Lec10：重构

Refactor

课程编号：22WHSE22006（2023版培养计划）
授课对象：23 级软件工程、服务科学与工程专业
主讲教师：辛国栋
Email：xinguodong@163.com



QQ 群：软件构造2025
群　号：311528424

本章内容

- 1. 重构四要素：包括重构的目的（why）、对象（what）、时机（when）、方法（how）；
- 2. 单元测试：保证重构不出错的有效手段
- 3. 代码的可测试性：如何编写可测试代码
- 4. 代码重构：那些方法可以用来解耦代码

1. 重构四要素：目的、对象、时机和方法

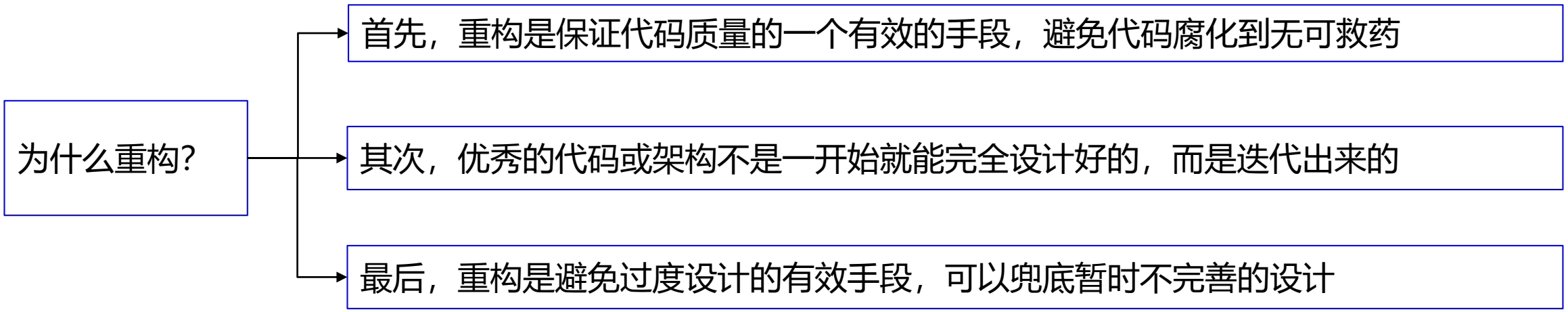
1.1 重构的目的：为什么要重构（why）？

软件设计大师 Martin Fowler 是这样定义重构的：“重构是一种对软件内部结构的改善，目的是在不改变软件的可见行为的情况下，使其更易理解，修改成本更低。”

- 定义中有一个值得强调的点：“重构不改变外部的可见行为”。

我们可以把重构理解为，在保持功能不变的前提下，利用设计思想、原则、模式、编程规范等理论来优化代码，修改设计上的不足，提高代码质量。

1.1 重构的目的：为什么要重构（why）？



- 重构对一个工程师本身技术的成长也有重要的意义：初级工程师在维护代码，高级工程师在设计代码，资深工程师在重构代码”，

1.2 重构的对象：到底重构什么（what）？

根据重构的规模，大致分为：

- 大型重构（大规模高层次）
  - 对顶层代码设计的重构，包括：系统、模块、代码结构、类与类之间的关系等的重构
  - 手段：分层、模块化、解耦、抽象可复用组件等
  - 重构的工具：设计思想、原则和模式
  - 特点：涉及的代码改动会比较多，影响面会比较大，所以难度也较大，耗时会比较长，引入 bug 的风险也会相对比较大
- 小型重构（小规模低层次）
  - 对代码细节的重构，主要是针对类、函数、变量等代码级别的重构，比如规范命名、规范注释、消除超大类或函数、提取重复代码等
  - 重构的工具：开发规范
  - 特点：要修改的地方比较集中，比较简单，可操作性较强，耗时比较短，引入 bug 的风险相对来说也会比较小。

1.3 重构的时机：什么时候重构（when）？

如同单元测试、Code Review 是开发的一部分；持续重构也是开发的一部分，成为一种开发习惯，并且在团队内形成共识。

技术在更新、需求在变化、人员在流动，代码质量总会在下降，代码总会存在不完美，重构就会持续在进行

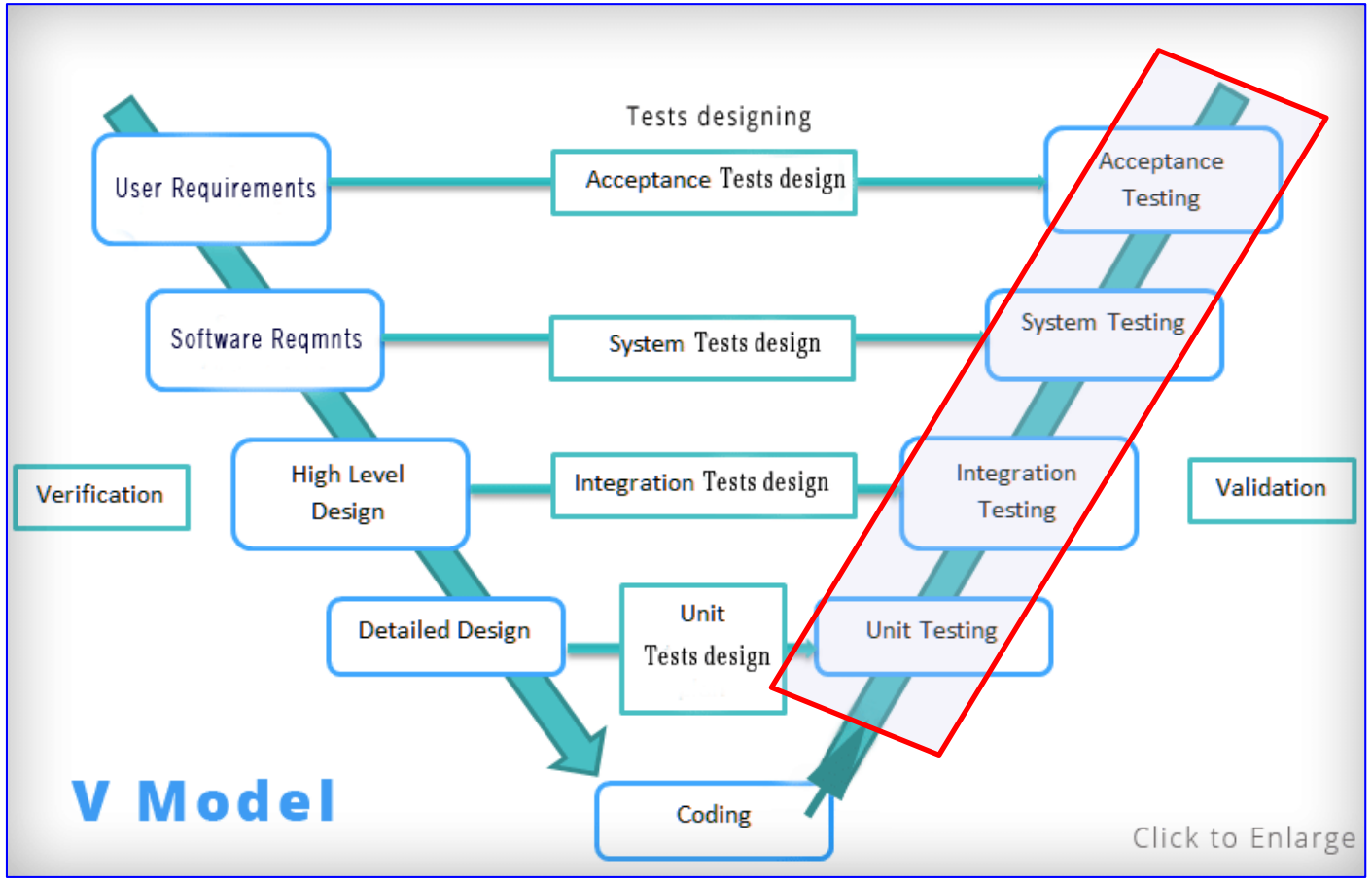
那些看到别人代码有点瑕疵就一顿乱骂，或者花尽心思去构思一个完美设计的人，往往都是因为没有树立正确的代码质量观，没有持续重构意识

1.4 重构的方法：如何重构（how）？

大规模高层次的重构	涉及的模块、代码比较多，如果项目代码质量比较差，耦合比较严重，往往会牵一发而动全身，本来觉得一天就能完成的重构，你会发现越改越多、越改越乱，没一两个礼拜都搞不定。而新的业务开发又与重构相冲突，最后只能半途而废，revert 掉所有的改动，很失落地又去堆砌烂代码了。
	一定是有组织、有计划，并且非常谨慎的，需要有经验、熟悉业务的资深同事来主导。
小规模低层次的重构	影响范围小，改动耗时短，所以，只要你愿意并且有时间，随时都可以去做
	可以借助很多成熟的静态代码分析工具（比如 CheckStyle、FindBugs、PMD），来自动发现代码中的问题，然后针对性地进行重构优化。

2. 单元测试：保证重构不出错的有效手段

2.1 软件测试的四个层次



验收测试：测试主题是需求方，其他与系统测试完全相同。

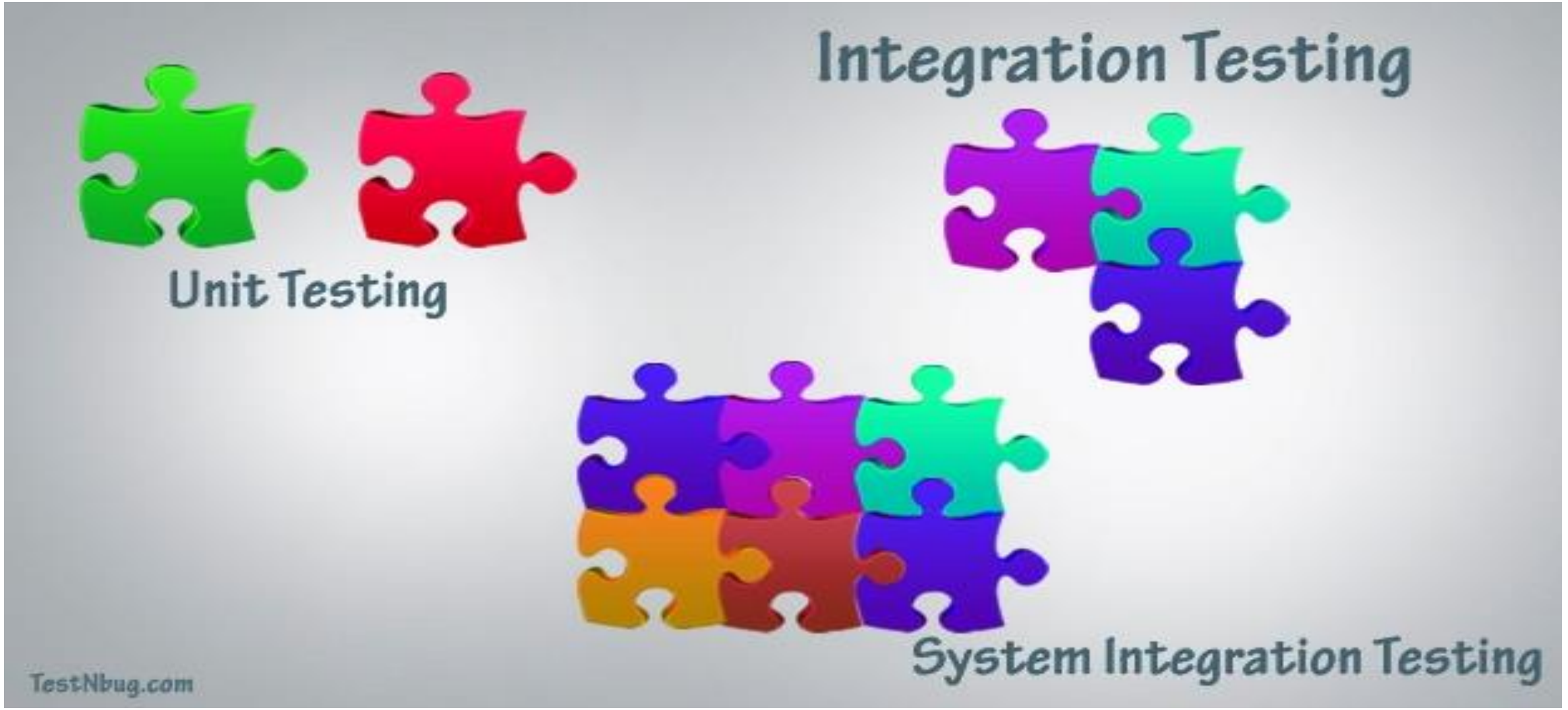
系统测试：测试一个完全集成的系统，以验证系统是否满足其要求，并在最终配置中执行该软件。

集成测试：由多个程序员或编程团队创建的两个或多个类，包，组件，子系统的组合执行。

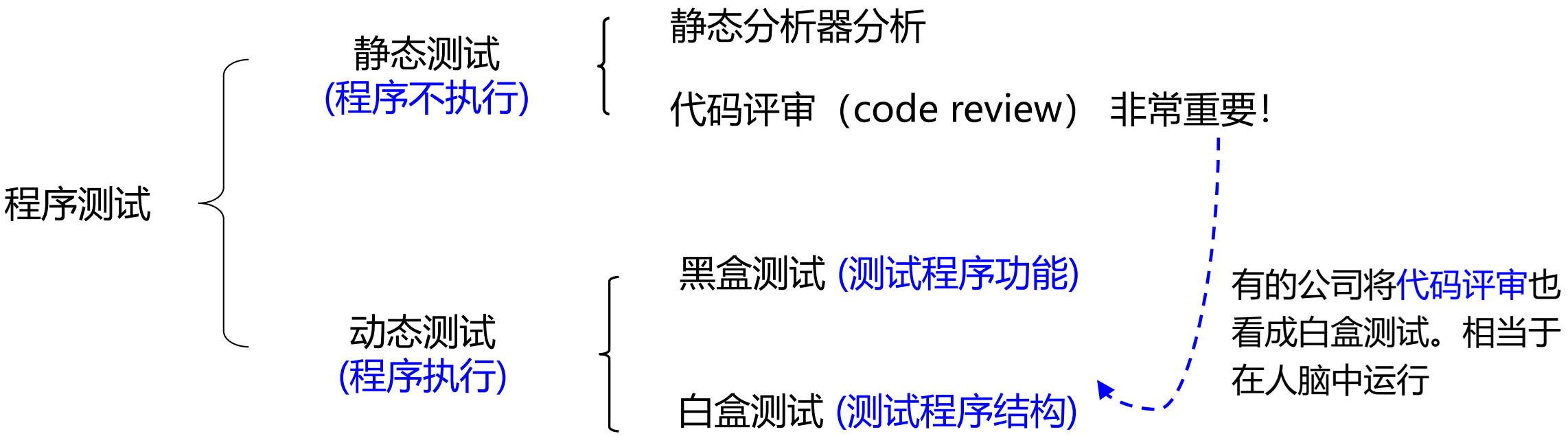
单元测试：指通常在功能级别（OO：类级别）验证特定代码段功能的测试。

测试过程必须分步骤进行，后一个步骤在逻辑上是前一个步骤的继续。

2.1 软件测试的四个层次

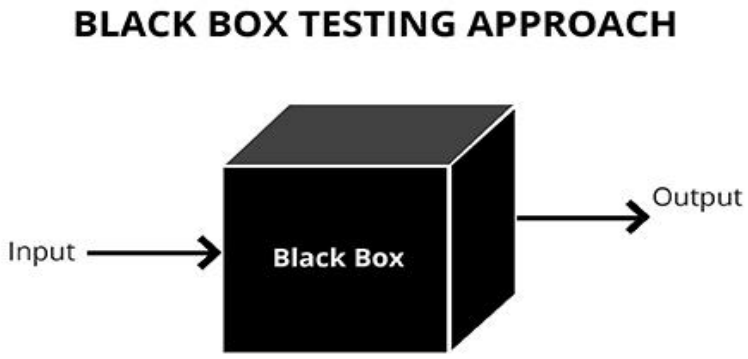


2.2 软件测试的时期

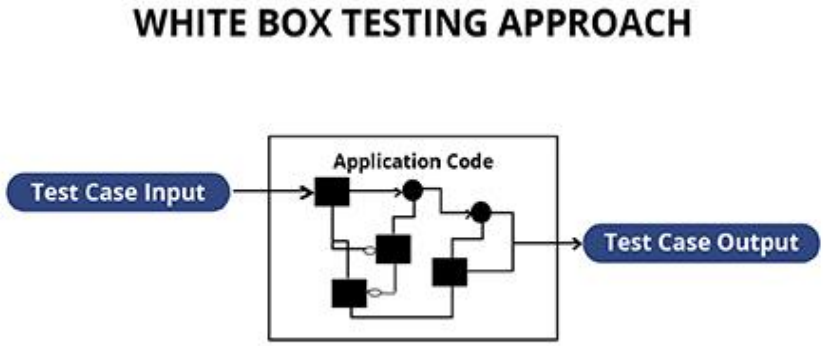


静态测试 vs 动态测试：靠眼睛看 vs 程序跑起来测

2.3 动态测试的两种核心方法

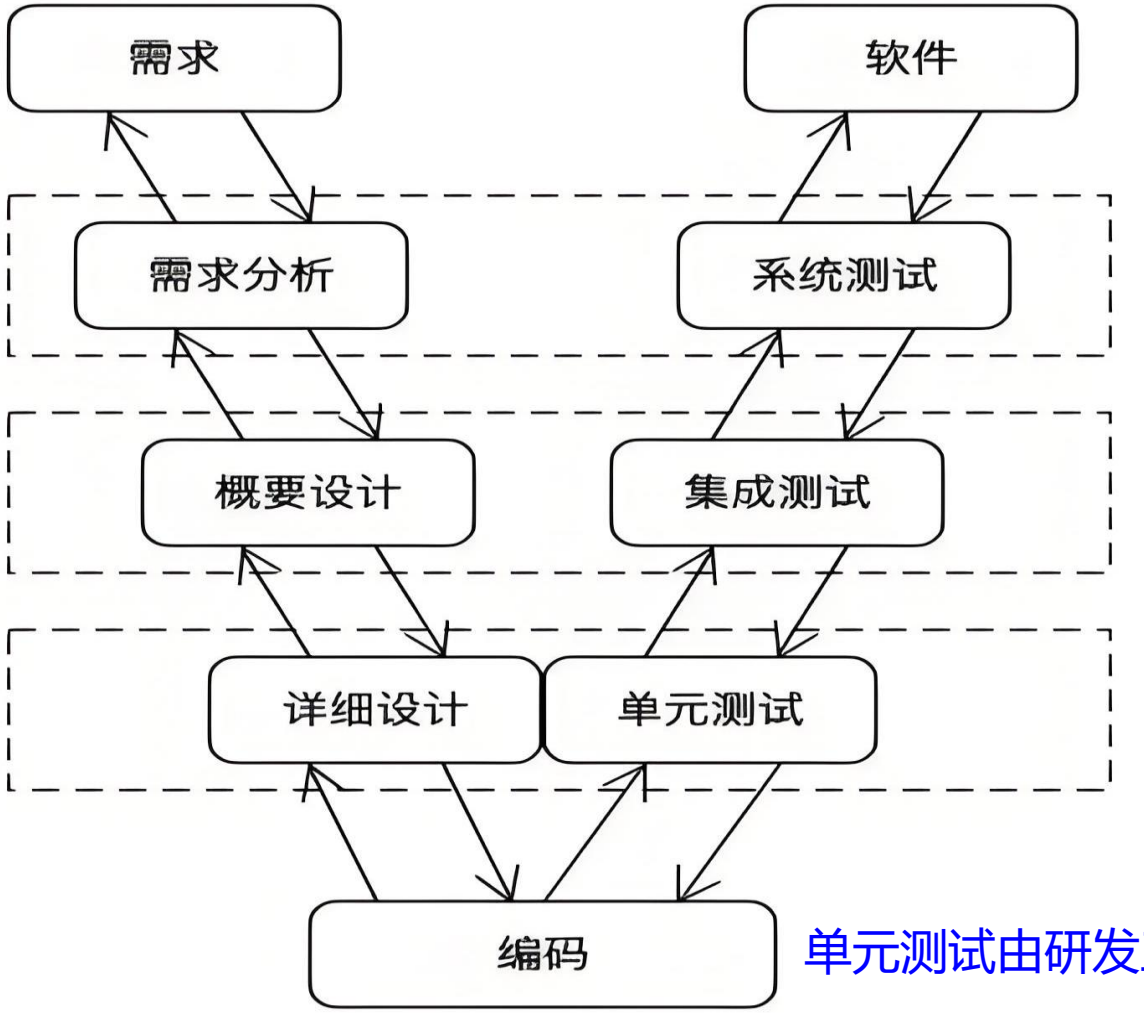


- 黑盒测试（又称功能测试）把程序看作一个黑盒子，完全不考虑程序的内部结构和处理过程。在程序接口进行的测试，测功能。
- 通俗来说即不知道代码是怎么写的,也不知道具体实现逻辑。只能根据代码输入有哪些应该输出什么进行测试



- 白盒测试（又称结构测试）是把程序看成装在一个透明的白盒子里，测试者完全知道程序的结构和处理算法。在程序内部的测试，测逻辑

2.3 动态测试的两种核心方法



采用黑盒测试。测功能。测试对象是整个系统或者某个功能模块。比如测试用户注册、登录功能是否正常，是一种端到端（end to end）的测试

采用白盒测试。测逻辑。单元测试的测试对象是类或者函数，用来测试一个类和函数是否都按照预期的逻辑执行。这是代码层级的测试

单元测试由研发工程师自己来编写，用来测试自己写的代码的正确性!

3. 代码的可测试性

3.1 什么是可测试性

- 代码的可测性（Testability）是衡量软件代码被测试的难易程度，即：一个软件系统或其组件在多大程度上支持测试。代码的可测性主要关注如何高效、全面地验证代码质量。
- 类比说明
  - 低可测试性设计：一辆汽车的引擎盖被焊死了，你无法直接看到或接触到任何内部零件。当汽车出现问题时，你唯一的办法就是听声音、闻气味，进行各种猜测。诊断问题既困难又耗时，而且极不准确。
  - 高可测试性设计：另一辆车的引擎盖可以轻松打开，所有关键部件（如火花塞、机油尺、电池）都有清晰的标记和便捷的检查端口。你可以使用标准工具轻松地测试每个部件的性能。诊断问题变得快速、精准且可靠。

- 低可测试性的代码就像那个被焊死的引擎，它的内部状态和依赖关系是隐藏的、耦合的，难以被观察和控制。高可测试性的代码则像那台易于维护的引擎，它的模块清晰、依赖明确，我们可以轻松地隔离任何一个部分进行测试。

3.1 什么是可测试性

```
public class ProductService {
    // 访问数据库的对象
     private ProduceRepository repository = new ProductRepository();
    public Product find(final long id) {
        return this.repository.find(id);
    }
}
```

测试该类有那些困难？

要创建数据库访问对象repository ，就要同时把数据库连接起来，要准备一大堆相关的东西，所以，测试的复杂度就会非常大

测试这个服务目的是看这个服务的逻辑是不是写正确了，与是不是用数据库没关系

因此修改为：

```
public class ProductService {
    // 访问数据库的对象
     private ProduceRepository repository;
    public ProductService(final ProduceRepository repository) {
        this.repository = repository;
    }

    public Product find(final long id) {
        return this.repository.find(id);
    }
}
```

只需要将数据访问的接口模拟出来，用来模拟接口的 Mock 框架在各种程序语言里几乎都可以找到。

保证模拟出来的对象要与接口定义的行为保持一致即可，不过，这可比准备数据库，难度系数要低多了

结论：可测试性要比修改前好。

3.3 典型常见的测试性不好的代码-未决行为

- 所谓的未决行为逻辑就是，代码的输出是随机或者说不确定的，比如，跟时间、随机数有关的代码，例如：

```
public class Demo {
    public long caculateDelayDays(Date dueTime) {
        long currentTimestamp = System.currentTimeMillis();
        if (dueTime.getTime() >= currentTimestamp) {
            return 0;
        }
        long delayTime = currentTimestamp - dueTime.getTime();
        long delayDays = delayTime / 86400;
        return delayDays;
    }
}
```

- 一般的处理方式是将这种未决行为逻辑重新封装

3.3 典型常见的测试性不好的代码-全局变量

- 全局变量是一种面向过程的编程风格，有种种弊端。实际上，滥用全局变量也让编写单元测试变得困难

```
public class RangeLimiter {
    private static AtomicInteger position = new AtomicInteger(0);
    public static final int MAX_LIMIT = 5;
    public static final int MIN_LIMIT = -5;

    public boolean move(int delta) {
        int currentPos = position.addAndGet(delta);
        boolean betweenRange = (currentPos <= MAX_LIMIT) && (currentPos >= MIN_LIMIT);
        return betweenRange;
    }
}
```

RangeLimiter 表示一个 [-5, 5] 的区间，position 初始在 0 位置，move() 函数负责移动 position。其中，position 是一个静态全局变量

3.3 典型常见的测试性不好的代码-全局变量

```
public class RangeLimiterTest {
    // 测试用例1
    public void testMove_betweenRange() {
        RangeLimiter rangeLimiter = new RangeLimiter();
        assertTrue(rangeLimiter.move(1));   // position 值变成 1 ：原来是 0，正向移动1
        assertTrue(rangeLimiter.move(3));   // position 值变成 4 ：原来是 1，正向移动 3
        assertTrue(rangeLimiter.move(-5));  // position 的值变成 -1：反向移动 -5
    }

    // 测试用例2
    public void testMove_exceedRange() {
        RangeLimiter rangeLimiter = new RangeLimiter();
        assertFalse(rangeLimiter.move(6)); // 静态变量 position = -1 正向移动 6，变成 5。期望超界
    }
}
```

上面的单元测试有可能会运行失败。假设单元测试框架顺序依次执行。在第一个测试用例执行完成之后，position 的值变成了 -1；再执行第二个测试用例的时候，position 变成了 5，move() 函数返回 true，assertFalse 语句判定失败（期望超界，但没超）。所以，第二个测试用例运行失败。

3.3 典型常见的测试性不好的代码-静态方法

- 静态方法跟全局变量一样，也是一种面向过程的编程思维。在代码中调用静态方法，有时候会导致代码不易测试。
- 主要原因是静态方法也很难 mock。

3.3 典型常见的测试性不好的代码-复杂继承

- 相比组合关系，继承关系的代码结构更加耦合、不灵活，更加不易扩展、不易维护。实际上，继承关系也更加难测试。
  - 如果父类需要 mock 某个依赖对象才能进行单元测试，那所有的子类、子类的子类……在编写单元测试的时候，都要 mock 这个依赖对象

- 利用组合而非继承来组织类之间的关系，类之间的结构层次比较扁平，在编写单元测试的时候，只需要 mock 类所组合依赖的对象即可。

3.3 典型常见的测试性不好的代码-高耦合代码

- 如果一个类职责很重，需要依赖十几个外部对象才能完成工作，代码高度耦合，那我们在编写单元测试的时候，可能需要 mock 这十几个依赖的对象。
- 不管是从代码设计的角度来说，还是从编写单元测试的角度来说，这都是不合理的。

4. 代码重构
Code Refactoring

4.1 代码重复 Duplicated Code：同类中多个方法间代码重复

Before

```
public class FruitsCost {
    public double computeMoneyWithoutPrivileges(String type, int numbers) {
        double prices;
        switch (type) {
            case "apple": prices = 5.5; break;
            case "banana": prices = 4.0; break;
            case "strawberry": prices = 10.5; break;
            default: throw new IllegalArgumentException("Illegal type : " + type);
        }
        return prices * numbers;
    }

    public double computeMoneyWithPrivileges(String type, int numbers, double discount) {
        double prices;
        switch (type) {
            case "apple": prices = 5.5; break;
            case "banana": prices = 4.0; break;
            case "strawberry": prices = 10.5; break;
            default: throw new IllegalArgumentException("Illegal type : " + type);
        }
        return prices * numbers * discount;
    }
}
```

抽取公共方法

4.1 代码重复 Duplicated Code：同类中多个方法间代码重复

After

```
public class FruitsCost {

    //提取水果价格计算方法
    private double getFruitPrice(String type) {
        switch (type) {
            case "apple": return 5.5;
            case "banana": return 4.0;
            case "strawberry": return 10.5;
            default: throw new IllegalArgumentException("Illegal type : " + type);
        }
    }

    //调用提取的方法
    public double computeMoneyWithoutPrivileges(String type, int numbers) {
        double price = getFruitPrice(type);
        return price * numbers;
    }

    public double computeMoneyWithPrivileges(String type, int numbers, double discount) {
        double price = getFruitPrice(type);
        return price * numbers * discount;
    }
}
```

抽取的公共方法

4.1 代码重复 Duplicated Code： 子类间代码重复

Before

```
public class Fruit {
    // 成本价
    protected double basePrice;
    // 出售单价
    protected double costPerUnit;
    // 最小出货量
    protected int minShipment;

    public Fruit(double basePrice, double costPerUnit, int minShipment) {
        this.basePrice = basePrice;
        this.costPerUnit = costPerUnit;
        this.minShipment = minShipment;
    }
}
```

Before

```java
public class Apple extends Fruit {
    public Apple() {
        super(5.5, 2.0, 100);
    }
    public double calculateProfit(int sales) {
        // 重复代码：计算有效销量(减去最小出货量)
        int effectiveSales = Math.max(sales - minShipment, 0);
        // 重复代码：计算毛利润(销售收入-成本)
        return effectiveSales * (basePrice - costPerUnit);
    }
}
```

Before

```java
public class Banana extends Fruit {
    public Banana() {
        super(4.0, 1.5, 50);
    }
    public double calculateProfit(int sales) {
        // 重复代码：计算有效销量(减去最小出货量)
        int effectiveSales = Math.max(sales - minShipment, 0);
        // 重复代码：计算毛利润(销售收入-成本)
        return effectiveSales * (basePrice - costPerUnit);
    }
}
```

相同代码。

解决方案：把相同代码上移到超类

After

```
public abstract class Fruit {
    protected double basePrice;
    protected double costPerUnit;
    protected int minShipment;

    public Fruit(double basePrice, double costPerUnit, int minShipment) {
        this.basePrice = basePrice;
        this.costPerUnit = costPerUnit;
        this.minShipment = minShipment;
    }

    // 提取公共逻辑：计算毛利润
    protected double calculateGrossProfit(int sales) {
        int effectiveSales = Math.max(sales - minShipment, 0);
        return effectiveSales * (basePrice - costPerUnit);
    }
}
```

```
public class Banana extends Fruit {
    public Banana() {
        super(4.0, 1.5, 50);
    }
}
```

After

```
public class Apple extends Fruit {
    public Apple() {
        super(5.5, 2.0, 100);
    }
}
```

After

4.1 代码重复 Duplicated Code： 不同类间的代码重复

Before

```java
public class MonthJudgement {
    public boolean judgeMonth() {
        long timeStamp = System.currentTimeMillis();
        SimpleDateFormat sdf = new SimpleDateFormat("yyyy/MM/dd");
        String date = sdf.format(new Date(Long.parseLong(String.valueOf(timeStamp))));
        String month =date.split(" ")[0].split("-")[1];
        return month.equals("12");
    }
}
```

代码逻辑不完全一致，变量名不同分别为time和timeState，judgeYear多了一条打印语句。

Before

```java
public class YearJudgement {
    public boolean judgeYear() {
        long time = System.currentTimeMillis();
        System.out.println("获取系统当前时间戳");
        SimpleDateFormat sdf = new SimpleDateFormat("yyyy/MM/dd");
        String date = sdf.format(new Date(Long.parseLong(String.valueOf(time))));
        return date.startsWith("2025");
    }
}
```

After

```java
public class DateFormatter {
    public static String getDate() {
        long timeStamp = System.currentTimeMillis();
        SimpleDateFormat sdf = new SimpleDateFormat("yyyy/MM/dd");
        return sdf.format(new Date(Long.parseLong(String.valueOf(timeStamp))));
    }
}
```

After

```java
public class MonthJudgement {
    public boolean judgeMonth() {
        String date = DateFormatter.getDate();
        String month =date.split(" ")[0].split("-")[1];
        return month.equals("12");
    }
}
```

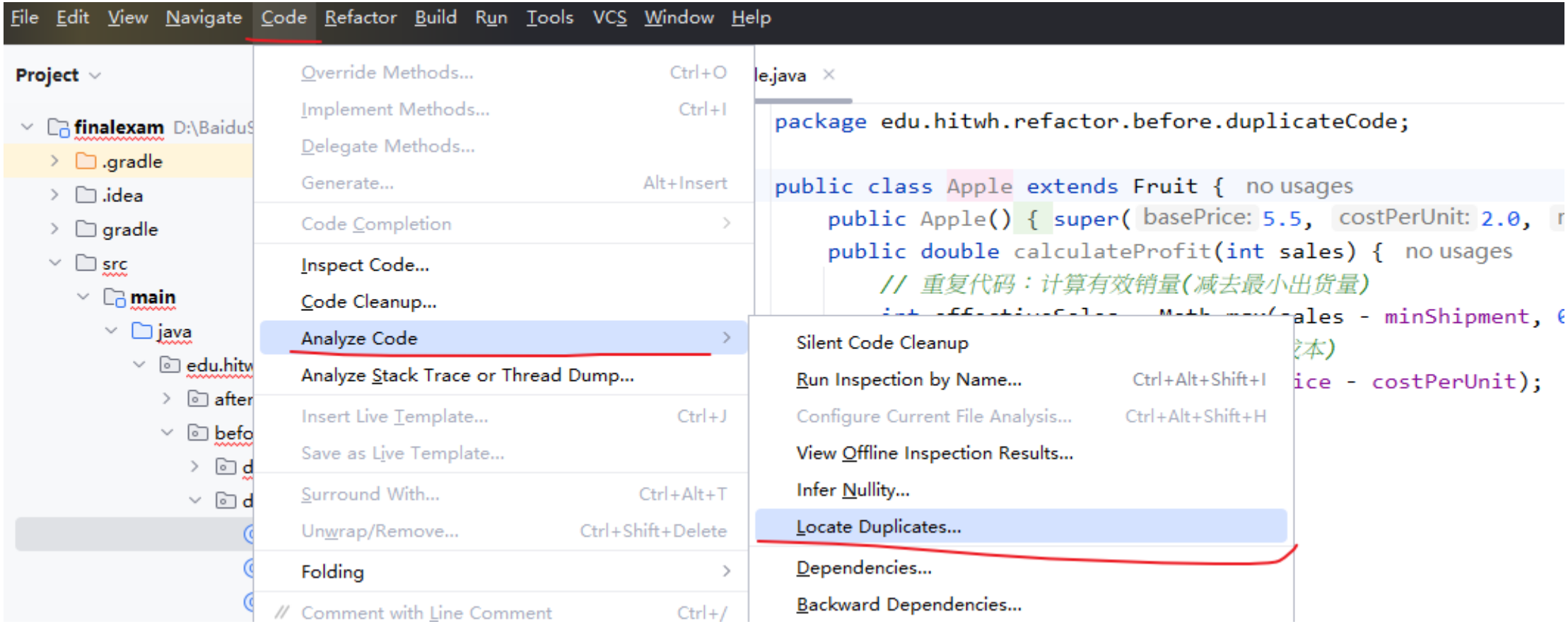
语句移动+抽取公共方法

After

```java
public class YearJudgement {
    public boolean judgeYear() {
        System.out.println("获取系统当前时间戳");
        String date = DateFormatter.getDate();
        return date.startsWith("2025");
    }
}
```

4.1 代码重复 Duplicated Code：识别重复代码的工具

- IDEA Analyze：IDEA对于重复代码会自动分析，并以波浪线标注出来。有良好的静态代码分析功能，例如修改变量名后，仍可被识别为重复代码



4.1 代码重复 Duplicated Code： 识别重复代码的工具

- 从重复具体的表现形态上，大致分为如下四类：

  - 类型Ⅰ：代码片段中除了空格、注释以及换行以外的内容完全一致
  - 类型Ⅱ：代码片段中除了空格、注释、换行以及变量名以外的内容完全一致
  - 类型Ⅲ：代码片段中除了空格、注释、换行以及变量名以外的语句可能有增删改，功能不变
  - 类型Ⅳ：两个或更多个代码段执行相同的运算，但通过不同的语法和变量来实现。

- 其中第四类属于逻辑、语法层面，需要结合人工经验判断，当前工具无法承载

4.1 代码重复 Duplicated Code： 识别重复代码的工具

名称	Simian	PMD-CMD	CloneDR	CCCD	CCFinder	Bauhaus	CodePro
检测类型	I	I,II	I,II	I,II	I	I,II,III	I,II,III
支持语言	C/C++/Java	C/C++/Java	C/C++/Java	C/C++/Java	C/C++/Java	C/C++/Java	Java
授权类型	商用 可免费	开源	商用 可免费	开源	开源	商用 可免费	开源
推荐星级	**	***	**	*	*	**	***

概念	定义	在一个以上的地点看到相同或相似的代码结构
	分类（基于分布位置）	同一个类的多个方法间代码重复
		互为兄弟的子类间代码重复
		不同类间的代码重复
	影响	难维护，想要修改其中一段代码逻辑需要修改多次，易遗漏
	目标	简化代码结构并使其更短
		简化+简短性=更容易简化且支持成本更低的代码
	改进方法	提炼函数，移动语句，函数上移
操作方法	提炼函数	Ctrl+Alt+M（推荐）
		Ctrl+Alt+Shift+T→Extract Method
		鼠标右键→refactor→Extract Method
	移动语句	Ctrl+Shift+↑/↓
	函数上移	Ctrl+Alt+Shift+T→Pull Members Up
		鼠标右键→refactor→Pull Members Up
	搬移函数 （注：使用快捷键需要先将方法变为静态方法）	F6(推荐)
		Ctrl+Alt+Shift+T→Move Instance Method
		鼠标右键→refactor→Move Instance Method
辅助工具	扫描插件及IDEA自带功能	CodeCheck，IDEA→Analyze
	其他工具	Simian/PMD-CMD/CloneDR/CCCD/CCFinder/Bauhaus/CodePro

4.2 全局数据 Global Data

Before

```
public class ClassStudentsInfo {
    public static Map<String, Students> classStudentsInfo = new HashMap<>();
    public static int classNumUpLimit = 3;
}
```

可以在系统中的任何位置对全局变量进行访问，并修改，甚至胡乱修改，导致数据泄露

Before

```
public class ClassManage {
    public void addClassInfo(String className,Students students) {
        if(className == null) {return;}
        if(classStudentsInfo.containsKey(className)) {return;}
        if(classStudentsInfo.size()>= classNumUpLimit){return;}

        classStudentsInfo.put(className,students);
        classStudentsInfo.put(className,new Students(new ArrayList<>())); //胡乱修改
    }
}
```

4.2 全局数据 Global Data

After

```
public class ClassStudentsInfo {
    private static Map<String, Students> CLASS_STUDENT_INFO = new HashMap<>();
    private static int CLASS_NUM_UP_LIMIT = 3;

    public void addStudents(String className, Students students) {
        // 校验代码
        CLASS_STUDENT_INFO.put(className, students);
    }
}
```

先把对全局变量的操作搬移到一个类中，然后把全局变量变为 private，将对 静态变量 的访问都封装在一个类中，从而消除全局变量；

4.2 全局数据 Global Data 小结

全局数据 Global Data	**概念**	定义	**类变量**和**单例**等可以在代码库的**任何一个角落修改**的数据	
		影响	可以在任何位置进行修改，在使用过程中可能出现**意想不到的值**，并且没有任何机制可以探测出哪段代码进行了修改，导致**定位困难**	
		目标	降低代码耦合性，保持**代码清晰，维护简单**，降低由于对全局数据随意的改变引发bug风险	
		改进方法	**封装变量，提炼函数**	
	改进手法	<<操作入口>>	快捷键（推荐）	Ctrl+Alt+Shift+T（或：鼠标右键“Refactor”、工具栏“Refactor”）
		提炼函数	Ctrl+Atl+M	Extract Method
		搬移静态成员	F6	Move Static Members
		转换为实例方法	N/A	Convert To Instance Method
		抽取字段	Ctrl+Alt+F	Introduce Field
	排查方法	搜索+人工判断		

4.3 可变数据 Mutable Data

Before

```
public class Invoice {
    private final String buyer;
    private final String seller;
    private List<InvoiceLine> lines;

    public Invoice(String buyer, String seller) {
        this.buyer = buyer;
        this.seller = seller;
    }

    public void setLine(InvoiceLine line) {this.lines.add(line);}

    public List<InvoiceLine> getLines() {return this.lines;}

    public BigDecimal getAmount() {return BigDecimal.ZERO;}
    public BigDecimal getTaxAmount() {return BigDecimal.ZERO;}

}
```

4.3 可变数据 Mutable Data

Before

```
public class InvoiceLine {
    private String productName;
    private BigDecimal unitPrice;
    private BigDecimal quantity;
    private BigDecimal taxRate;

    private BigDecimal Amount;
    private BigDecimal taxAmount;

    public InvoiceLine() { }
    public InvoiceLine(String productName, BigDecimal unitPrice, BigDecimal quantity, BigDecimal taxRate) {
        this.productName = productName;
        this.unitPrice = unitPrice;
        this.quantity = quantity;
        this.taxRate = taxRate;
    }

    public void setQuantity(BigDecimal quantity) {this.quantity = quantity;}
    public void setUnitPrice(BigDecimal unitPrice) {this.unitPrice = unitPrice;}
}
```

4.3 可变数据 Mutable Data

Before

```
public class Invoice {
    private final String buyer;
    private final String seller;
    private List<InvoiceLine> lines;

    public Invoice(String buyer, String seller) {
        this.buyer = buyer;
        this.seller = seller;
    }

    public void setLine(InvoiceLine line) {this.lines.add(line);}

    public List<InvoiceLine> getLines() {return this.lines;}

    public BigDecimal getAmount() {return BigDecimal.ZERO;}
    public BigDecimal getTaxAmount() {return BigDecimal.ZERO;}

}
```

看到集合类，就需要考虑是否有封装泄露的问题

有setter，可改变数据

有对外暴露内部字段的可变引用

4.3 可变数据 Mutable Data

Before

```
public class InvoiceLine {
    private String productName;
    private BigDecimal unitPrice;
    private BigDecimal quantity;
    private BigDecimal taxRate;

    private BigDecimal Amount;
    private BigDecimal taxAmount;

    public InvoiceLine() { }
    public InvoiceLine(String productName, BigDecimal unitPrice, BigDecimal quantity, BigDecimal taxRate) {
        this.productName = productName;
        this.unitPrice = unitPrice;
        this.quantity = quantity;
        this.taxRate = taxRate;
    }
    public void setQuantity(BigDecimal quantity) {this.quantity = quantity;}
    public void setUnitPrice(BigDecimal unitPrice) {this.unitPrice = unitPrice;}
    //getter略
}
```

多余，可以通过上面的变量计算出来

有setter，可改变数据

4.3 可变数据 Mutable Data

```
public class Invoice {
    private final String buyer;
    private final String seller;
    private final List<InvoiceLine> lines;

    public Invoice(String buyer, String seller, List<InvoiceLine> lines) {
        this.buyer = buyer;
        this.seller = seller;
        this.lines = lines;
    }

    public void setLine(InvoiceLine line) {this.lines.add(line);}

    public List<InvoiceLine> getLines() { return Collections.unmodifiableList(lines);}

    public BigDecimal getAmount() {return BigDecimal.ZERO;}
    public BigDecimal getTaxAmount() {return BigDecimal.ZERO;}

}
```

加 final 变成私有的常量

改为构造函数注入

删除 setter

封装可变类型，并且要求每个集合内部的元素不可变

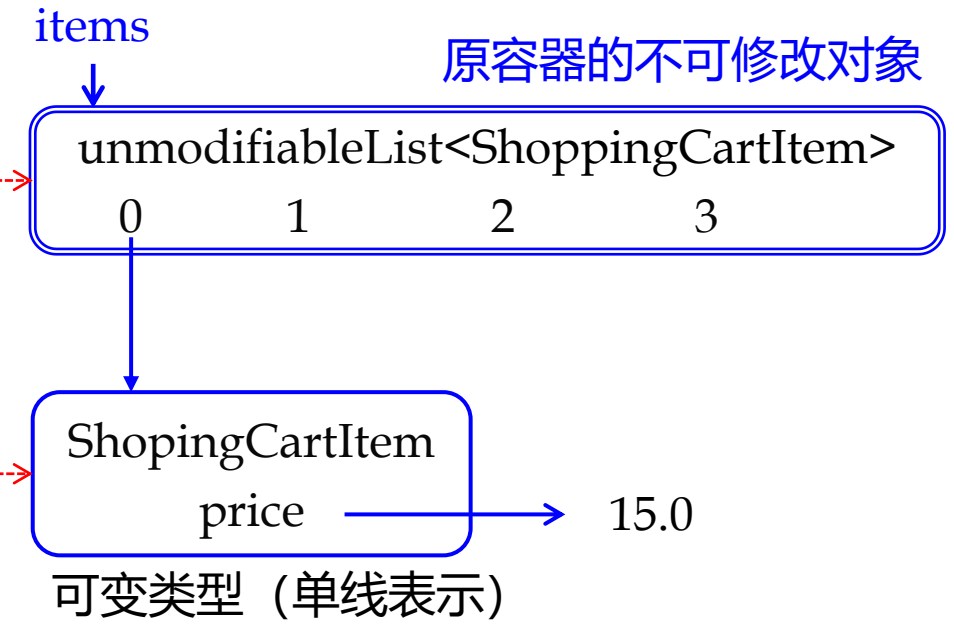
复习：容器本身不可变，但容器内的数据是可变的

```java
public class ShoppingCart {
    // 省略其他代码
    public List<ShoppingCartItem> getItems() {
        // return this.items;
        return Collections.unmodifiableList(this.items);
    }
}
```

```java
public class ShoppingCartTest {
    @Test
    public void testDataLeak(){
        ShoppingCart cart = new ShoppingCart();
        cart.addItem(new ShoppingCartItem());
        List<ShoppingCartItem> items = cart.getItems();

        ShoppingCartItem item = items.get(0);
        item.setPrice(15.0);
        System.out.println(item.getPrice());
    }
}
```

快照图（Snapshot Diagraph）。其中双线表示不可修改对象



items本身不可修改，但 items中存储的对象可以被修改，并没有解决数据泄露问题。

课程的后续内容将讨论解决方案

4.3 可变数据 Mutable Data

```
public class InvoiceLine {
    private final String productName;
    private final BigDecimal unitPrice;
    private final BigDecimal quantity;
    private final BigDecimal taxRate;

    private BigDecimal Amount;
    private BigDecimal taxAmount;

    public InvoiceLine() { }
    public InvoiceLine(String productName, BigDecimal unitPrice, BigDecimal quantity, BigDecimal taxRate) {
        this.productName = productName;
        this.unitPrice = unitPrice;
        this.quantity = quantity;
        this.taxRate = taxRate;
    }

    public void setQuantity(BigDecimal quantity) {this.quantity = quantity;}
    public void setUnitPrice(BigDecimal unitPrice) {this.unitPrice = unitPrice;}
    // getter略
}
```

如果InvoiceLine不可变，则Invoice中的List<InvoiceLine> lines有了不可变封装，就安全了。

删除

删除

删除

4.3 可变数据 Mutable Data

After

```
public class Invoice {
    private final String buyer;
    private final String seller;
    private final List<InvoiceLine> lines;

    public Invoice(String buyer, String seller, List<InvoiceLine> lines) {
        this.buyer = buyer;
        this.seller = seller;
        this.lines = lines;
    }

    public List<InvoiceLine> getLines() {
        return Collections.unmodifiableList(lines);
    }
    public BigDecimal getAmount() {return BigDecimal.ZERO;}
    public BigDecimal getTaxAmount() {return BigDecimal.ZERO;}
}
```

4.3 可变数据 Mutable Data

After

```
public class InvoiceLine {
    private final String productName;
    private final BigDecimal unitPrice;
    private final BigDecimal quantity;
    private final BigDecimal taxRate;

    public InvoiceLine(String productName, BigDecimal unitPrice, BigDecimal quantity, BigDecimal taxRate) {
        this.productName = productName;
        this.unitPrice = unitPrice;
        this.quantity = quantity;
        this.taxRate = taxRate;
    }
    // getter 省略...
}
```

4.3 可变数据 Mutable Data 小结

<table>
<tr><td rowspan="9">可变数据
Mutable
Data</td><td rowspan="4">概念</td><td colspan="2">定义</td><td colspan="2">对数据的修改经常导致出乎意料的结果和难以发现的Bug。
注：并非所有可变类型都是不良的，这里关注描述数据的可变类型</td></tr>
<tr><td colspan="2" rowspan="2">影响</td><td colspan="2">影响可维护性，对它的修改经常导致出乎意料的结果和难以发现的bug</td></tr>
<tr><td colspan="2">在一处修改数据，却在另一处造成难以发现的破坏</td></tr>
<tr><td colspan="2">改进目标</td><td colspan="2">应用“数据不可变”：不可变性是强大的代码防腐剂</td></tr>
<tr><td colspan="2">改进方法</td><td colspan="2">封装变量、拆分变量、提炼函数、移除设值函数、查询取代派生、Builder模式创建不可变对象、引用对象改为值对象、函数式编程等</td></tr>
<tr><td rowspan="4">症状</td><td colspan="4">一个变量用作不同目的</td></tr>
<tr><td colspan="4">数据类型中，用public修饰非final成员</td></tr>
<tr><td colspan="4">有set方法（或其它改变数据的方法）</td></tr>
<tr><td colspan="4">一个成员变量的值可通过其他字段计算得到</td></tr>
<tr><td rowspan="5">改进手法
（仅列主要方法）</td><td colspan="4">对外暴露内部变量的可变引用</td></tr>
<tr><td colspan="2"><<操作入口>></td><td>快捷键（推荐）</td><td>Ctrl+Alt+Shift+T（或：鼠标右键“Refactor”、工具栏“Refactor”）</td></tr>
<tr><td colspan="2">封装变量</td><td>N/A</td><td>Encapsulate Fields</td></tr>
<tr><td colspan="2">拆分变量（用提取函数）</td><td>Ctrl+Atl+M</td><td>Extract Method</td></tr>
<tr><td rowspan="2">移除设值函数</td><td>用内联移除</td><td>Ctrl+Alt+N</td><td>Inline Method</td></tr>
<tr><td>直接删除</td><td>Alt+Del</td><td>Safe Delete</td></tr>
<tr><td colspan="2">Builder模式创建不可变对象</td><td>N/A</td><td>Replace Constructor as Builder</td></tr>
</table>

4.4 过长函数 Long Function

```
public class InvoicePrinter {
    public static void print(Invoice invoice, PrintStream printStream) {
        printStream.println("====== INVOICE ======");
        printStream.println("Buyer: " + invoice.getBuyer());
        printStream.println("Seller: " + invoice.getSeller());

        printStream.println("------ Detail ------");
        printStream.println("ln\tProduct\tPrice\tQt\tAmount");
        float total = 0;
        for (int lineId = 0; lineId < invoice.getLines().size(); ++lineId) {
            final InvoiceLine line = invoice.getLines().get(lineId);
            String product = line.getProduct();
            float price = line.getPrice();
            float quantity = line.getQuantity();
            float amount = price * quantity;
            if (product == null || "".equals(product.trim()) || price <= 0 || quantity <= 0) {
                printStream.println(lineId + "\tInvalid");
            } else {
                printStream.println(lineId + "\t" + product + "\t" + price + "\t" + quantity + "\t" + amount);
            }
            total += amount;
        }
        printStream.println("------ Total ------");
        printStream.println(total);
    }
}
```

```
====== INVOICE ======
Buyer: buyer
Seller: seller
------ Detail ------
ln  Product Price   Qt  Amount
0   a   10.0    10.0    100.0
1   b   20.0    2.0 40.0
------ Total ------
140.0
```

理解长函数功能意图，归纳功能流程，提炼功能步骤

4.4 过长函数 Long Function

```
====== INVOICE ======
Buyer: buyer
Seller: seller
------ Detail ------
ln  Product Price   Qt  Amount
0   a   10.0    10.0    100.0
1   b   20.0    2.0 40.0
------ Total ------
140.0
```

```java
public class InvoicePrinter {
    public static void print(Invoice invoice, PrintStream printStream) {
        printStream.println("====== INVOICE ======");
        printStream.println("Buyer: " + invoice.getBuyer());
        printStream.println("Seller: " + invoice.getSeller());

        printStream.println("------ Detail ------");
        printStream.println("ln\tProduct\tPrice\tQt\tAmount");
        float total = 0;
        for (int lineId = 0; lineId < invoice.getLines().size(); ++lineId) {
            final InvoiceLine line = invoice.getLines().get(lineId);
            String product = line.getProduct();
            float price = line.getPrice();
            float quantity = line.getQuantity();
            float amount = price * quantity;
            if (product == null || "".equals(product.trim()) || price <= 0 || quantity <= 0) {
                printStream.println(lineId + "\tInvalid");
            } else {
                printStream.println(lineId + "\t" + product + "\t" + price + "\t" + quantity + "\t" + amount);
            }
            total += amount;
        }
        printStream.println("------ Total ------");
        printStream.println(total);
    }
}
```

打印发票头

打印发票体

打印发票表的头部

打印发票发票行

打印发票尾

4.4 过长函数 Long Function

```
for (int lineId = 0; lineId < invoice.getLines().size(); ++lineId) {
    final InvoiceLine line = invoice.getLines().get(lineId);
    String product = line.getProduct();
    float price = line.getPrice();
    float quantity = line.getQuantity();
    float amount = price * quantity;
    if (product == null || "".equals(product.trim()) || price <= 0 || quantity <= 0) {
        printStream.println(lineId + "\tInvalid");
    } else {
        printStream.println(lineId + "\t" + product + "\t" + price + "\t" + quantity + "\t" + amount);
    }
}
printStream.println("------ Total ------");
```

打印发票体

```
float total = 0;
for (int lineId = 0; lineId < invoice.getLines().size(); ++lineId) {
    final InvoiceLine line = invoice.getLines().get(lineId);
    String product = line.getProduct();
    float price = line.getPrice();
    float quantity = line.getQuantity();
    float amount = price * quantity;
    total += amount;
}
printStream.println(total);
```

打印发票尾

Total 需要从原来的循环中分离出来

```java
public static void print(Invoice invoice, PrintStream printStream) {
    printStream.println("====== INVOICE ======");
    printStream.println("Buyer: " + invoice.getBuyer());
    printStream.println("Seller: " + invoice.getSeller());

    printStream.println("------ Detail ------");
    printStream.println("ln\tProduct\tPrice\tQt\tAmount");
    for (int lineId = 0; lineId < invoice.getLines().size(); ++lineId) {
        final InvoiceLine line = invoice.getLines().get(lineId);
        String product = line.getProduct();
        float price = line.getPrice();
        float quantity = line.getQuantity();
        float amount = price * quantity;
        if (product == null || "".equals(product.trim())
                || price <= 0 || quantity <= 0) {
            printStream.println(lineId + "\tInvalid");
        } else {
            printStream.println(lineId + "\t" + product + "\t"
                    + price + "\t" + quantity + "\t" + amount);
        }
    }

    printStream.println("------ Total ------");
    float total = 0;
    for (int lineId = 0; lineId < invoice.getLines().size(); ++lineId) {
        final InvoiceLine line = invoice.getLines().get(lineId);
        float price = line.getPrice();
        float quantity = line.getQuantity();
        float amount = price * quantity;
        total += amount;
    }
    printStream.println(total);
}
```

```java
ic class InvoicePrinter {
public static void print(Invoice invoice, PrintStream printStream) {
    printHeader(invoice, printStream);

    printDetail(invoice, printStream);

    printFooter(invoice, printStream);
}

private static void printFooter(Invoice invoice, PrintStream printStream) {
    printStream.println("------ Total ------");
    float total = 0;
    for (int lineId = 0; lineId < invoice.getLines().size(); ++lineId) {...}
    printStream.println(total);
}

private static void printDetail(Invoice invoice, PrintStream printStream) {
    printStream.println("------ Detail ------");
    printStream.println("ln\tProduct\tPrice\tQt\tAmount");
    for (int lineId = 0; lineId < invoice.getLines().size(); ++lineId) {...}
}

private static void printHeader(Invoice invoice, PrintStream printStream) {
    printStream.println("====== INVOICE ======");
    printStream.println("Buyer: " + invoice.getBuyer());
    printStream.println("Seller: " + invoice.getSeller());
}
```

提炼函数抽象主函数流程

注：以“做什么”命名提炼的函数，而不是“怎么做”

选中对应的代码段，在IDEA中使用Ctrl+Shift+M,快捷重构

4.4 过长函数 Long Function

- 复杂语句下沉到子函数，对复杂子函数递归重构– printDetail 函数

```
private static void printDetail(Invoice invoice, PrintStream printStream) {
    printStream.println("------ Detail ------");
    printStream.println("ln\tProduct\tPrice\tQt\tAmount");
    for (int lineId = 0; lineId < invoice.getLines().size(); ++lineId) {
        final InvoiceLine line = invoice.getLines().get(lineId);
        String product = line.getProduct();
        float price = line.getPrice();
        float quantity = line.getQuantity();
        float amount = price * quantity;
        if (product == null || "".equals(product.trim()) || price <= 0 || quantity <= 0) {
            printStream.println(lineId + "\tInvalid");
        } else {
            printStream.println(lineId + "\t" + product + "\t" + price + "\t"
                + quantity + "\t" + amount);
        }
    }
}
```

printTableHeader

printInvoiceLines

```java
private static void printDetail(Invoice invoice, PrintStream printStream) {
    printTableHeader(printStream);
    printInvoiceLines(invoice, printStream);
}

private static void printInvoiceLines(Invoice invoice, PrintStream printStream) {
    for (int lineId = 0; lineId < invoice.getLines().size(); ++lineId) {
        printInvoiceLine(invoice, printStream, lineId);
    }
}

private static void printInvoiceLine(Invoice invoice, PrintStream printStream, int lineId) {
    final InvoiceLine line = invoice.getLines().get(lineId);
    String product = line.getProduct();
    float price = line.getPrice();
    float quantity = line.getQuantity();
    float amount = price * quantity;
    if (product == null || "".equals(product.trim()) || price <= 0 || quantity <= 0) {
        printStream.println(lineId + "\tInvalid");
    } else {
        printStream.println(lineId + "\t" + product + "\t"
                + price + "\t" + quantity + "\t" + amount);
    }
}
```

```
private static void printDetail(Invoice invoice, PrintStream printStream) {
    printTableHeader(printStream);
    printInvoiceLines(invoice, printStream);
}

private static void printInvoiceLines(Invoice invoice, PrintStream printStream) {
    for (int lineId = 0; lineId < invoice.getLines().size(); ++lineId) {
        printInvoiceLine(invoice, printStream, lineId);
    }
}

private static void printInvoiceLine(Invoice invoice, PrintStream printStream, int lineId) {
    final InvoiceLine line = invoice.getLines().get(lineId);

    String product = line.getProduct();
    float price = line.getPrice();
    float quantity = line.getQuantity();

    float amount = price * quantity;

    if (product == null || "".equals(product.trim()) || price <= 0 || quantity <= 0) {
        printStream.println(lineId + "\tInvalid");
    } else {
        printStream.println(lineId + "\t" + product + "\t"
                + price + "\t" + quantity + "\t" + amount);
    }
}
```

还存在问题，例如临时变量过多：应该以查询取代变量

功能内聚到合适的位置，并以查询取代变量

条件过多，可以分解条件语句

4.4 过长函数 Long Function

过长函数 Long Function	**概念**	定义	一个函数包含了**过多的逻辑功能或过分体现逻辑功能的实现细节**，导致函数产生过长的代码块	
		影响	过长的函数往往意味着函数功能不单一；或过分呈现细节未进行抽象。造成**可读性差**和**增加代码维护成本问题**；	
			过长函数还容易发生资源未释放等编码问题	
		目标	抽象待重构函数流程，分解出子函数，最大化精简出**可读性强、功能单一的函数**	
		改进方法	**提炼函数**：找到函数中适合集中在一起的部分，将他们提炼出来形成一个新函数，并用函数**“做什么”来命名新的函数**	
	症状	函数过长		
		函数功能不单一		
		业务功能实现过分呈现细节		
	改进手法（仅列主要方法）	<<操作入口>>	快捷键（推荐）	Ctrl+Alt+Shift+T（或：鼠标右键“Refactor”、工具栏“Refactor”）
		提炼函数	Ctrl+Atl+m	Extract Method
		移动函数	F6	Move Instance Method
		以查询取代变量	Ctrl+Atl+n	Inline Variable
	排查方法	搜索+人工判断	CodeCheck 扫描出超大函数优先进行重构	
	辅助工具/参考资料	Code Check	辅助定位超大函数	

4.5 发散式变化（Divergent Change）

- 定义：某个模块经常因为不同的原因在不同的方向上发生变化（一个类中，一处代码的修改影响多个功能，称作发散式变化。）

- 影响：通常，发散式变化是由于多个变化方向之间有较多的来回调用或者函数内部混合了多类处理逻辑。当外部行为发生变化时候，都会引起对同一个类或模块的修改，影响了代码的可读和可维护性。

- 改进目标：提高代码组织结构、职责单一提升代码可读性、可维护性

- 方法：拆分阶段、搬移函数、提炼函数、提炼类

发散式变化 Divergent Change	**概念**	定义	某个类经常因为**不同的原因**在**不同的方向**上发生**变化**	
		影响	通常，发散式变化是由于多个变化方向之间有较多的来回调用或者函数内部混合了多类处理逻辑。当处于**多个**不同上下文的**外部行为发生变化**时候，都会**引起对同一个类或模块的修改，影响了代码的可读和可维护性**	
		目标	**提高代码组织结构、减少重复代码**提升代码可读性、可维护性	
		改进方法	**拆分阶段、搬移函数、提炼函数、提炼类**	
	症状	多个变化方向之间有较多的来回调用		
		模块内部混合了多类处理逻辑		
	改进手法	<<操作入口>>	快捷键（推荐）	Ctrl+Alt+Shift+T（或：鼠标右键“Refactor”、工具栏“Refactor”）
		提炼函数	Ctrl+Alt+M	Extract Method
		实例/静态方法搬移	F6	Move Instance Method/Move Static Members
		内联	Ctrl+Alt+N	Inline XXX
		实例方法变为静态方法	N/A	Make Static
		静态方法变为实例方法	N/A	Convert To Instance Method
		引入参数对象	N/A	Introduce Parameter Object
		提取字段/参数/变量	Ctrl+Alt+F/P/V	Introduce Field/Parameter/Variable
		提炼接口	N/A	Extract Interface

4.6 霰弹式修改 Shotgun Surgery

- 定义：霰弹式修改：指的是如果每遇到某种变化，你都必须在许多不同的类内做出许多小修改。此时，你所面临的坏味道就是霰弹式修改（一个功能的变更需要多处同时修改，比如，订单和用户服务里都有日志方法，改日志就得动两处。）

- 影响：需要修改的代码散布四处，你不但很难找到它们，也很容易错过某个重要的修改。任何修改遗漏都会导致异常结果

- 改进目标：更好的代码组织；更少的代码重复；更易维护

- 方法：搬移函数、搬移字段、函数组合成类、函数组合成变换、拆分阶段、内联函数、内联类

4.6 霰弹式修改 Shotgun Surgery

霰弹式修改 (Shotgun Surgery)	**概念**	定义	指的是如果每遇到某种变化，你都必须在许多不同的类内做出许多小修改	
		影响	需要修改的代码散布四处，你不但很难找到它们，也很容易错过某个重要的修改	
		目标	更好的代码组织、更少的代码重复，极大提升代码的可维护性及可扩展性	
		改进方法	搬移函数、搬移字段、函数组合成类、函数组合成变换、拆分阶段、内联函数、内联类	
	症状	某些类似或者关联操作多个类或方法中		
	改进手法 (仅列主要方法)	<<操作入口>>	快捷键（推荐）	Ctrl+Alt+Shift+T（或：鼠标右键“Refactor”、工具栏“Refactor”）
		提炼常量	Ctrl+Atl+C	Introduce Constant
		提炼函数	Ctrl+Atl+M	Extract Method
		搬移实例方法/静态成员	F6	Move Instance Method/Move Static Members
		内联	Ctrl+Alt+N	Inline XXX
	排查方法	搜索+人工判断	Analyze->Inspect Code 分析Duplicated code fragment 片段	
			人工识别	
	辅助工具/参考资料	《重改-改善既有代码设计》第2版	3.8节	

4.7 基本类型偏执 Primitive Obsession

```
public class Plane {
    private final Logger logger;
    public Plane(Logger logger) { this.logger = logger;}

    public void fly(List<int[]> airLine, String airLineType) {
        for (int[] coordinate : airLine) {
            if ( “domestic” .equals(airLineType)) { // 国内航班
                 if (coordinate[0] < 0 || coordinate[0] >= 100) { //抛异常 }
                if (coordinate[1] < 0 || coordinate[1] >= 100) {//抛异常 }
            }
            if ("international".equals(airLineType)) { // 国际航班
                 if (coordinate[0] < 100) {//抛异常 }
                 if (coordinate[1] < 100) {//抛异常 }
            }
        }
        airLine.forEach(this::fly);
    }

    private void fly(int[] coordinate) {
        logger.info(MessageFormat.format("Fly to ({0},{1})", coordinate[0], coordinate[1]));
    }
}
```

描述了飞机按照航线类型飞行的业务；

- 航线包含坐标集合、类型；
- 航线坐标包含2个int数值；
- 飞行前需要校验航线，不同航线类型校验内容不同

- 内聚性、可读性差，对外暴露了内部细节，如坐标的数据结构、航线校验逻辑，代码层次不清晰
- 通过航线类型字符串区分不同航班执行不同校验逻辑，后续增加航线类型时需要修改fly方法，方法会越来越复杂，可维护性差

4.7 基本类型偏执 Primitive Obsession

```
public class Plane {
    private final Logger logger;
    public Plane(Logger logger) { this.logger = logger;}

    public void fly(List<int[]> airLine, String airLineType) {
        for (int[] coordinate : airLine) {
            if ( “domestic” .equals(airLineType)) { // 国内航班
                if (coordinate[0] < 0 || coordinate[0] >= 100) { //抛异常 }
                if (coordinate[1] < 0 || coordinate[1] >= 100) {//抛异常 }
            }
            if ("international".equals(airLineType)) { // 国际航班
                if (coordinate[0] < 100) {//抛异常 }
                if (coordinate[1] < 100) {//抛异常 }
            }
        }
        airLine.forEach(this::fly);
    }

    private void fly(int[] coordinate) {
        logger.info(MessageFormat.format("Fly to ({0},{1})", coordinate[0], coordinate[1]));
    }
}
```

需提炼航线对象

需提炼航线接口，并用多态取代表达式

需提炼坐标对象

需封装细节

4.7 基本类型偏执 Primitive Obsession

- 定义: 对于具有意义的业务概念如钱、坐标、范围等，不愿意进行建模，而是使用基本数据类型进行表示
- 影响: 暴露较多细节，代码内聚性差，可读性差
- 改进目标: 消除基本类型，提升代码可修改性、内聚性、可读性
- 方法:
  - 1、对象取代基本类型
  - 2、子类取代类型
  - 3、多态取代条件表达式
  - 4、提炼类
  - 5、引入参数对象

4.7 基本类型偏执 Primitive Obsession

基本类型偏执 Primitive Obesession	**概念**	定义	对于具有意义的业务概念如钱、坐标、范围等，不愿意进行建模，而是使用基本数据类型进行表示	
		影响	暴露较多细节，**代码内聚性差，可读性差**	
		目标	**消除基本类型**，提升代码可修改性、内聚性、可读性	
		改进方法	1、对象取代基本类型 2、子类取代类型 3、多态取代条件表达式 4、提炼类 5、引入参数对象	
	症状	使用基本类型而非对象		
		暴露过多细节未提炼封装		
		字符串的条件表达式		
	改进手法（仅列主要方法）	<<操作入口>>		快捷键（推荐）
		引入参数对象		N/A
		子类取代类型码	创建子类	Alt+Enter -> Create subclass
			工厂方法代替构造方法	N/A
		多态取代表达式	提取接口	N/A
			上推成员方法、变量	N/A
			下推成员方法、变量	N/A

Ctrl+Alt+Shift+T（或：鼠标右键“Refactor”、工具栏“Refactor”）
Introduce Parameter Object
N/A
Replace Constructor With Factory Method
Extract Interface
Pull Member Up
Push Member Down

4.8 数据泥团（Data Clumps）

```
public class PersonInfoManage {
    private String firstName;
    private String lastName;
    private Gender gender;
    private String province;
    private String city;
    private String street;

    public PersonInfoManage(String firstName, String lastName, Gender gender
       , String province, String city, String street) {
    }

    public String getName() { }
    public Gender getGender() { }
    public String getAddress() { }
    public void updateAddress(String province, String city, String street) { }
    private void doNotify(String province, String city) {    }
    public String moveToAnotherPlace(String newProvince, String newCity, String newStreet) {   }
}
```

一个人员信息管理类，包含了人员的姓名、性别、地址等信息，以及人员的信息获取、地址更新、位置移动等相关处理方法

4.8 数据泥团（Data Clumps）

```
public class PersonInfoManage {
    private String firstName;
    private String lastName;
    private Gender gender;
    private String province;
    private String city;
    private String street;

    public PersonInfoManage(String firstName, String lastName, Gender gender
        , String province, String city, String street) {
    }

    public String getName() { }
    public Gender getGender() { }
    public String getAddress() { }
    public void updateAddress(String province, String city, String street) { }
    private void doNotify(String province, String city) {    }
    public String moveToAnotherPlace(String newProvince, String newCity, String newStreet) {    }
}
```

- 问题：
  - ✓ 用户的名字(firstName,lastName)和所在的位置(provice,city,street)是没有直接关联的数据，但都以基本类型罗列在一起，导致参数过长
  - ✓ 成块的参数总是重复出现(provice,city,street)

- 重构路线：
  - ✓ 将名字和地址抽取为独立的类，拆解不同领域；通过搬移方法将函数组合成类，使不同领域的数据与各自行为封装在一起

4.8 数据泥团（Data Clumps）

```
public class PersonInfoManage {
    private final Name name;
    private final Gender gender;
    private final Address address;

    public PersonInfoManage(Name name, Gender gender, Address address) {
        this.name = name;
        this.gender = gender;
        this.address = address;
    }

    public String getName() { }
    public Gender getGender() { }
    public String getAddress() { }
    public void updateAddress(Address newAddress) {
        address.updateAddress(newAddress);
    }
    public String moveToAnotherPlace(Address anotherAddress) {  }
}
```

通过引入参数对象、保持对象完整性

```java
public class Name {
    private final String firstName;
    private final String lastName;
    public Name(String firstName, String lastName) {
        this.firstName = firstName;
        this.lastName = lastName;
    }
    public String printName() {}
}
```

```java
public class Address {
    private String province;
    private String city;
    private String street;

    public Address() {}
    public Address(String province, String city, String street) {}
    public String getAddress() { }
    public void updateAddress(Address anotherAddress) { }
    private void doNotify() {    }
    public String moveToAnotherPlace(Address anotherAddress) {    }
}
```

通过搬移方法将函数组合成类，使不同领域的数据与各自行为封装在一起

4.8 数据泥团（Data Clumps）

数据泥团 Data Clumps	**概念**	定义	总是**成块出现的相同数据项**，包括多个类中相同的字段、多个方法签名中相同的参数等	
		影响	成块出现的重复参数过多，影响阅读和理解，**难维护**	
		目标	减少相同的字段及入参，**缩短**入参列，**简化**函数调用	
		改进方法	**提炼类、引入参数对象、保持对象完整性**	
	症状	成块的参数总是重复出现在多个类字段、方法签名中		
		方法参数过多[1]		
	改进手法	<<操作入口>>	快捷键（推荐）	Ctrl+Alt+Shift+T（或：鼠标右键“Refactor”、工具栏“Refactor”）
		提炼类	N/A	Extract Delegate
		引入参数对象	N/A	Introduce Parameter Object
		保持对象完整性（通过提炼和内联组合实现） 提炼函数	Ctrl+Alt+M	Extract Method
		保持对象完整性（通过提炼和内联组合实现） 内联	Ctrl+Alt+N	Inline XXX
		实例方法搬移	F6	Move Instance Method
		抽取变量/字段/参数	Ctrl+Alt+V/F/P	Introduce Variable/Field/Parameter
	排查方法	人工排查	删掉众多数据中的一部分，观察其他数据有没有因失去意义。如果它们不再有意义，应该为它们产生一个新对象	
		工具辅助	对于工具扫描出超过5个的参数的方法，有数据泥团的概率更，可以优先排查[2]	

4.9 依恋情节（Feature Envy）

```
public class BankAccount {
    private int accountAge;
    private int creditScore;
    private AccountInterest accountInterest;
    public BankAccount(int accountAge, int creditScore, AccountInterest accountInterest) {}

    public int getAccountAge() {}
    public int getCreditScore() {}

    public AccountInterest getAccountInterest() {}
    public double calculateInterestRate() {}
}
```

AccountInterest使用calculateInterestRate()方法更为频繁，它更希望得到该方法，

因此，我们需要成人之美，将calculateInterestRate()方法从BankAccount类搬移到AccountInterest类中

```
public class AccountInterest {
    private BankAccount account;
    public AccountInterest(BankAccount account) {   }
    public BankAccount getAccount() {  }
    public double getInterestRate() {  return account.calculateInterestRate(); }
    public boolean isIntroductoryRate() {   return (account.calculateInterestRate() < 0.05); }
}
```

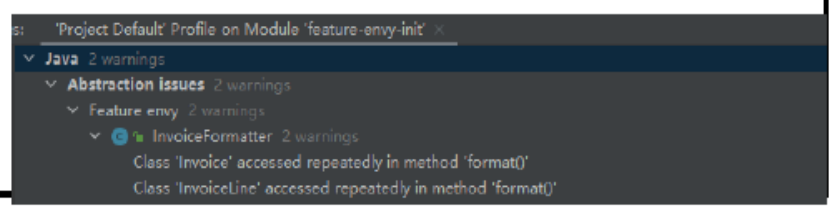
4.9 依恋情节（Feature Envy）

```
public class BankAccount {
    private int accountAge;
    private int creditScore;
    private AccountInterest accountInterest;
    public BankAccount(int accountAge, int creditScore, AccountInterest accountInterest) {}
    public int getAccountAge() { return this.accountAge;   }
    public int getCreditScore() { return this.creditScore;     }
    public AccountInterest getAccountInterest() {   return this.accountInterest; }
}
```

```
public class AccountInterest {
    private BankAccount account;
    public AccountInterest(BankAccount account) {  this.account = account;   }
    public BankAccount getAccount() {return this.account;  }
    public double getInterestRate() {  return calculateInterestRate(); }
    public boolean isIntroductoryRate() {   return (calculateInterestRate() < 0.05); }
    //将calculateInterestRate()方法从BankAccount类搬移到AccountInterest类
     public double calculateInterestRate() {  }
}
```

4.9 依恋情节（Feature Envy）

<table>
<tr><td rowspan="6">依恋情节
Feature Envy</td><td rowspan="4">概念</td><td>定义</td><td colspan="3">一个函数跟另一个模块中的函数或数据交流格外频繁，远胜于在自己所处模块内部的交流。
注：策略模式、访问者模式往往会带来依恋情节，我们需要权衡不同代码形态的收益</td></tr>
<tr><td rowspan="2">影响</td><td colspan="3">可读性、可维护性低，使用另一模块功能时往往需要打一套组合拳才能完成，需要知道很多被调用者的细节。</td></tr>
<tr><td colspan="3">往往还会伴随有重复代码、霰弹式修改、内幕交易……</td></tr>
<tr><td>改进目标</td><td colspan="3">将函数搬移到对应的类，解除跨模块的过多交流</td></tr>
<tr><td>改进方法</td><td colspan="3">提炼函数、搬移函数</td></tr>
<tr><td>症状</td><td colspan="4">在一个函数中对另一对象有频繁的组合调用，却对自身的访问却偏少。</td></tr>
<tr><td rowspan="3">改进手法
（仅列主要方法）</td><td><<操作入口>></td><td>快捷键（推荐）</td><td colspan="2">Ctrl+Alt+Shift+T（或：鼠标右键“Refactor”、工具栏“Refactor”）</td></tr>
<tr><td>提取函数</td><td>Ctrl+Atl+M</td><td colspan="2">Extract Method</td></tr>
<tr><td>移动函数</td><td>F6</td><td colspan="2">Move Instance Method</td></tr>
<tr><td>辅助工具</td><td>IDEA “Inspect Code”</td><td colspan="3">需勾选配置项：
Setting -> Editor -> Inspections ->
Java -> Abstraction issues -> Feature envy
功能入口：
IDEA -> Analyze -> Inspect Code</td></tr>
</table>



4.10 过长参数列表（Long Parameter List）

```
public class TicketInfo {
    private final double baseDiscount;
    public TicketInfo(double baseDiscount) { this.baseDiscount = baseDiscount;}
    /**
     * 获取票据信息
     *
     * @param name 姓名
     * @param age 年龄
     * @param isChild 是否儿童
     * @param isStudent 是否学生
     * @param ageFloor 年龄上限
     * @param ageCeiling 年龄下限
     * @param performance 演出信息
     * @param basicPrice 基本票价
     * @return 票据信息
     */
    public String getTicketInfo(String name, int age, boolean isChild, boolean isStudent
        , int ageFloor, int ageCeiling,Performance performance, double basicPrice) {}
    //其它方法略...
}
```

4.10 过长参数列表（Long Parameter List）

过长参数列表 Long Parameter List	**概念**	定义	方法的入参过多	
		影响	方法不易被理解、使用	
			方法签名容易**不稳定**	
		目标	去除多余参数，使用对象代替若干个参数，减少方法参数个数，提升方法签名稳定性	
		改进方法	查询取代参数、保持对象完整、引入参数对象、移动函数至适合的类、移除标记参数等	
	症状	某些入参可通过其他入参计算得到——**查询取代参数**		
		某几个入参是一个对象的部分字段——**保持对象完整**		
		某几个入参有关联性——**引入参数对象**		
		函数逻辑都是针对某个入参对象属性的加工——**移动函数至适合的类**		
		某些入参属于标记，用于控制代码逻辑——**移除标记参数**		
	改进手法（仅列主要方法）	<<操作入口>>	快捷键（推荐）	Ctrl+Alt+Shift+T（或：鼠标右键“Refactor”、工具栏“Refactor”）
		删除无用入参	Alt+Del/Alt+Enter	NA
		添加入参	Ctrl+F6/Alt+Enter	Change Signature
		提取参数对象	NA	Parameter Object
		抽取方法	Ctrl+Alt+M	Method
		内联方法	Ctrl+Alt+N	Inline Method
		移动方法	F6	Move Method
	排查方法	搜索+人工判断	CodeCheck扫描出超过5个入参的方法，优先进行重构	
	辅助工具/参考资料	Code Check	帮助搜索出超过5个入参的方法	

4.11 神秘命名（Mysterious Name）

```
public class MysteriousExample {

    public int amont(Perf perf, Type type) {
        switch (type) {
            case TYPE1:
                return resfortype1(perf);
            case TYPE2:
                return resfortype2(perf);
            default:
                throw new IllegalArgumentException("Illegal type : " + type);
        }
    }


    private int resfortype1(Perf perf) {}
    private int resfortype2(Perf perf) {}
}
```

4.11 神秘命名（Mysterious Name）

神秘命名 Mysterious Name	**概念**	定义	在阅读代码时，一些会影响我们的阅读体验的神秘文字（不知其意的命名）	
		影响	影响可读性，未来在“猜谜”上可能浪费大把时间	
			如果代码难以命名，其背后可能潜藏更深的设计问题	
		目标	代码直观明了	
			函数、模块、变量和类命名能清晰地表名自己的功能和用法	
		改进方法	改变函数声明（用于给函数改名）、变量改名、字段改名等	
	改进手法	<<操作入口>>	快捷键（推荐）	Ctrl+Alt+Shift+T（或：鼠标右键“Refactor”、工具栏“Refactor”）
		重命名	Shift+F6	Rename
		执行当前光标所在的可执行代码	Ctrl+Shift+F10	N/A
		执行上次运行的代码	Shift+F10	N/A
	辅助工具/参考资料	IDEA-Analyze-Inspect Code	可辅助识别拼写错误、不当缩写	

4.12 重复的 Switch（Repeated Switches）

```
public class Employee {
    public static final int ENGINEER = 0;
    public static final int SALESMAN = 1;
    private int type;
    public Employee(int type) {this.type = type;}
    public int calculatePay() {
        switch (type) {
            case ENGINEER:
                return 5000;
            case SALESMAN:
                return 4000;
            default:
                throw new IllegalArgumentException("Incorrect Employee Type");
        }
    }
    public int getBonus() {
        switch (type) {
            case ENGINEER:
                return 1000;
            case SALESMAN:
                return 1500;
            default:
                throw new IllegalArgumentException("Incorrect Employee Type");
        }
    }
}
```

4.12 重复的 Switch（Repeated Switches）

```
public abstract class Employee {
    public abstract int calculatePay();
    public abstract int getBonus();
}
```

```
public class Engineer extends Employee {
    @Override
    public int calculatePay() { return 5000; }

    @Override
    public int getBonus() { return 1000; }
}
```

```
public class Salesman extends Employee {
    @Override
    public int calculatePay() {  return 4000; }

    @Override
    public int getBonus() { return 1500;}
}
```

4.12 重复的 Switch（Repeated Switches）

```
public abstract class EmployeeFactory {
    public abstract Employee createEmployee() ;
}
```

```
public class EngineerFactory extends EmployeeFactory{
    @Override
    public Employee createEmployee() {
        return new Engineer();
    }
}
```

```
public class SalesFactory extends EmployeeFactory{
    @Override
    public Employee createEmployee() {
        return new Salesman();
    }
}
```

4.12 重复的 Switch（Repeated Switches）

```
public class Main {
    public static void main(String[] args) {
        EmployeeFactory salessFactory = new SalesFactory();
        Employee employee = salessFactory.createEmployee();
        int i = employee.calculatePay();
        int bonus = employee.getBonus();
        System.out.println("pay: "+ i +" bonus: "+ bonus);

        System.out.println("================================");
        EmployeeFactory engineerFactory = new EngineerFactory();
        Employee engineer = engineerFactory.createEmployee();
        i = engineer.calculatePay();
        bonus = engineer.getBonus();
        System.out.println("pay: "+ i +" bonus: "+ bonus);
    }
}
```

4.12 重复的Switch（Repeated Switches）

重复的Switch Repeated Switches	**概念**	定义	在不同的地方反复使用**同样的switch逻辑**	
		影响	**影响可维护性**：每当需要增加一个选择分支时，必须找到所有的switch，并逐一更新	
		目标	消除重复switch，**提升代码可修改/可扩展能力**	
		改进方法	**多态取代条件表达式**	
	症状	同样的switch逻辑反复出现		
	改进手法	<<操作入口>>	快捷键（推荐）	Ctrl+Alt+Shift+T（或：鼠标右键“Refactor”、工具栏“Refactor”）
		创建子类	Alt+Enter→Create SubClass	N/A
		用工厂替换构造方法	N/A	Replace Constructor With Factory Method
		多态返回子类对象	N/A（需手动编写）	N/A（需手动编写）
		方法搬移	F6	Extract Method
		静态方法转换为实例方法	N/A	Convert To Instance Method
		方法下移到子类	N/A	Push Members down
		修改visibility	Alt+Enter→Make xxx protected/public/.....	N/A
	排查方法	搜索+人工判断	全局搜索switch，看重复的判断条件是否反复出现（有时switch会以if-else形态出现，也需要关注）	
	修改建议	多态+工厂	参考设计模式——工厂模式	
		多态+提供不同实现类	参考List——ArrayList、LinkedList等	

4.13 循环语句（Loops）

- 定义：针对集合或数组进行简单过滤、分组、映射、遍历操作，采用传统for循环语句进行遍历
- 影响：影响代码复杂度，代码不够简洁、直观，不容易理解代码意图。
- 改进目标：针对可以使用stream替代的for循环，使用stream替代，简化代码
- 方法：针对集合或数组过滤、映射、去重、查找、统计使用stream替代

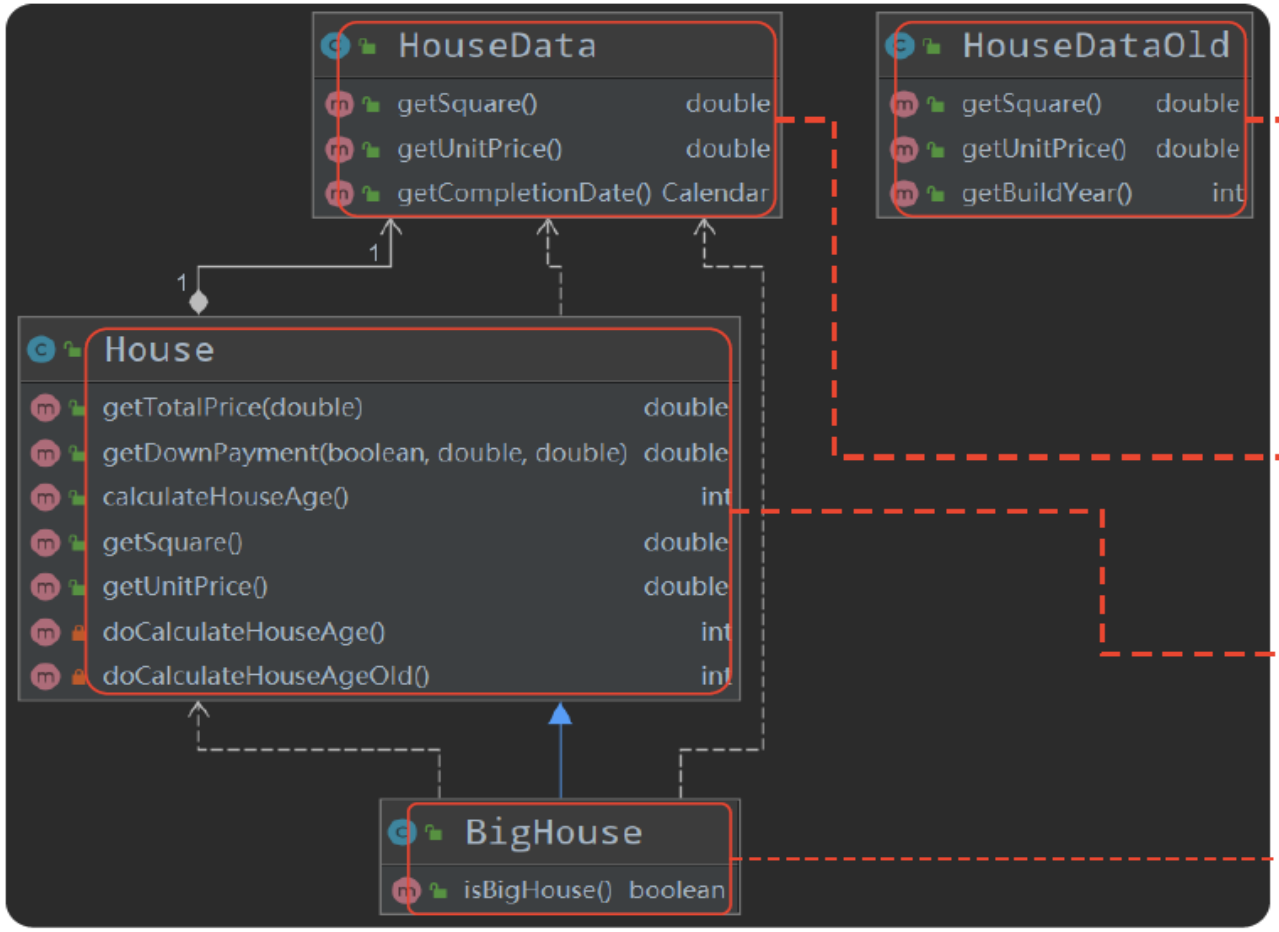
- 注：并非所有for循环都是坏味道。for循环作为极其常见的语法，本身没有问题，只是有些对集合、数组遍历的操作，for循环实现比较复杂，可以使用更简洁的stream表达式进行替换

4.13 循环语句（Loops）

<table>
<tr><td rowspan="10">循环语句
Loops</td><td rowspan="4">概念</td><td colspan="2">定义</td><td>针对集合或数组进行简单分组、过滤、映射、遍历操作，采用传统for循环语句进行遍历
(注：并非所有for都是不良的，基本功能可以使用stream更简洁)</td></tr>
<tr><td colspan="2">影响</td><td>影响代码复杂度，代码不够简洁、直观，不容易理解代码意图。</td></tr>
<tr><td colspan="2">目标</td><td>针对可以使用Stream替代的for循环，简化代码，也更直观易懂</td></tr>
<tr><td colspan="2">改进方法</td><td>针对集合或数组分组、过滤、映射、去重、查找、统计使用Stream替代</td></tr>
<tr><td>症状</td><td colspan="3">采用for循环实现简单分组、过滤、映射、去重、查找、统计等功能</td></tr>
<tr><td rowspan="6">操作方法</td><td><<操作入口>></td><td>快捷键（推荐）</td><td>Ctrl+Alt+Shift+T（或：鼠标右键“Refactor”、工具栏“Refactor”）</td></tr>
<tr><td>基于IDEA能力重构</td><td>ALT+ENTER——根据提示选择替换方法（如Replace with sum()、Replace with collect()等）</td><td>N/A</td></tr>
<tr><td>内联</td><td>Ctrl+Alt+N</td><td>Inline xxx</td></tr>
<tr><td>调整代码格式和换行</td><td>Ctrl+Alt+L</td><td>N/A</td></tr>
<tr><td>执行当前代码</td><td>Ctrl+Shift+F10</td><td>N/A</td></tr>
<tr><td>执行上一次运行的代码</td><td>Shift+F10</td><td>N/A</td></tr>
<tr><td>排查方法</td><td colspan="2">搜索+人工判断</td><td>全局搜索for，针对数组、集合遍历操作，看遍历操作是否基本为过滤、分组、映射等逻辑功能</td></tr>
</table>

4.14 冗赘的元素（Lazy Element）

代码结构



代码背景

- 简化的房屋信息处理模型，可为购房者提供房屋面积、单价、总价、首付款、房龄等相关信息

旧版本系统使用的房屋数据模型，当前业务中未使用

当前系统使用的房屋数据模型，包含单价、面积、竣工时间，以及对应的get方法

封装了房屋数据处理方法的类

专门处理大房子信息的类，设计之初曾拥有很多职责，演进过程中其他功能逐渐删除，当前只剩判断是否是大房子的方法

4.14 冗赘的元素（Lazy Element）

- 存在旧版系统中使用的类、方法、字段，当前版本既然已确定不再使用，就应当及时删除
- 出于对“未来可能使用”的考虑引入的参数，但当前实际代码中并不会使用，应当删除，避免歧义

4.14 冗赘的元素（Lazy Element）

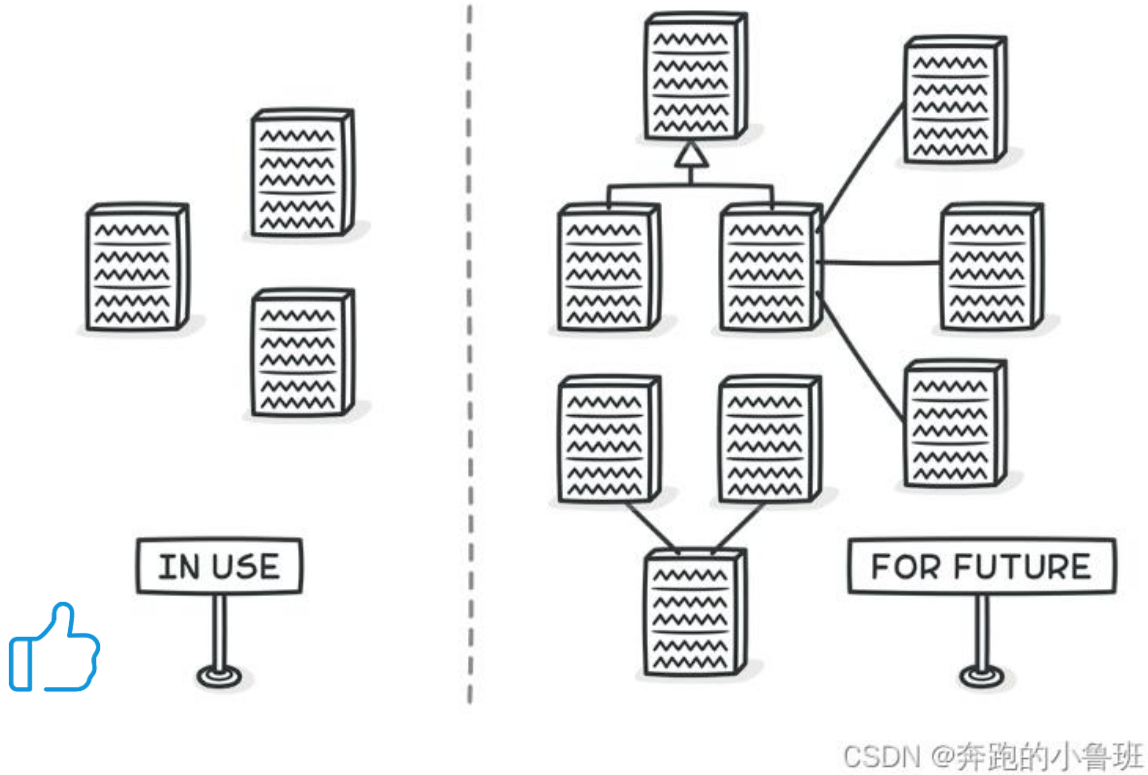
- 定义：冗赘的元素主要包括由于过度设计或在代码演进过程中，产生的冗余、废弃或不足以独立承担其职责的类、方法、变量等
- 影响：代码不简洁，存在多余的元素，造成在维护时无用修改，难以维护，影响代码的可读性
- 改进目标：消除冗赘的程序元素，提高代码的可读性、可维护性。

4.14 冗赘的元素（Lazy Element）

基本类型偏执 Primitive Obsession	**概念**	定义	冗赘的元素主要包括由于**过度设计**或在**代码演进**过程中，产生的**冗余、废弃**[1]**或不足以独立承担其职责**的类、方法、变量等	
		影响	代码不简洁，存在多余的元素，造成在维护时**无用修改**，**难以维护**，影响代码的可读性。	
		目标	**消除冗赘**的程序元素，提高代码的**可读性**、**可维护性**。	
		改进方法	**内联函数**或**内联类**	
			如果这个类处于一个继承体系中，可以使用**折叠继承体系**	
			安全删除冗余元素	
	症状	被注释掉的代码、未被使用的参数、变量、函数、类		
		函数或方法过于简单，不足以独立承担其职责		
	改进手法（仅列主要方法）	<<操作入口>>	快捷键（推荐）	Ctrl+Alt+Shift+T（或：鼠标右键“Refactor”、工具栏“Refactor”）
		封装变量	N/A	Encapsulate Fields
		提炼参数	Ctrl+Atl+P	Introduce Parameter
		提炼字段	Ctrl+Atl+F	Introduce Field
		用内联移除	Ctrl+Shift+N	Inline xxx
		安全删除	Alt+Del	Safe Delete
	排查方法	工具辅助+人工判断	代码上的废弃和冗余，可以使用Cmetrics和IDEA的Analyze功能进行辅助排查	
			设计上的冗赘元素，需要结合人工经验进行排查	
	辅助工具	IDEA-Analyze-Inspect Code	可辅助识别未使用的类、函数、变量、参数	
		Cmetrics	可以扫描出被注释掉的代码	

4.15 夸夸其谈的通用性（Speculative Generality）

- 定义：过度的考虑程序的通用性
- 影响：过度的设计导致代码不易理解和维护
- 改进目标：删除过度设计的代码
- 方法
  - 折叠继承体系
  - 内联类
  - 内联函数
  - 改变函数声明
  - 移除死代码



CSDN @奔跑的小鲁班

4.15 夸夸其谈的通用性（Speculative Generality）

<table>
<tr><td rowspan="10">夸夸其谈的通用性
Speculative Generality</td><td rowspan="4">概念</td><td>定义</td><td colspan="2">过度的考虑程序的通用性</td></tr>
<tr><td>影响</td><td colspan="2">代码可读性差、不易维护</td></tr>
<tr><td>目标</td><td colspan="2">删除过度设计的代码</td></tr>
<tr><td>改进方法</td><td colspan="2">折叠继承体系、内联类、内联函数、改变函数声明、移除死代码</td></tr>
<tr><td rowspan="4">症状</td><td colspan="3">抽象类和接口其实没有太大作用——折叠继承体系</td></tr>
<tr><td colspan="3">不必要的委托——内联类、内联函数</td></tr>
<tr><td colspan="3">函数的参数并未被使用——改变函数声明</td></tr>
<tr><td colspan="3">函数只被测试用例调用或者未被调用——移除死代码</td></tr>
<tr><td rowspan="3">改进手法</td><td><<操作入口>></td><td>快捷键（推荐）</td><td>Ctrl+Alt+Shift+T（或：鼠标右键“Refactor”、工具栏“Refactor”）</td></tr>
<tr><td>移除超类</td><td>Ctrl+Alt+N</td><td>Inline Super Class</td></tr>
<tr><td>删除参数</td><td>Alt+Delete</td><td>Safe Delete</td></tr>
<tr><td>辅助工具</td><td>IDEA-Analyze-Inspect Code</td><td colspan="2">可辅助识别冗余的参数、方法、类和接口等</td></tr>
</table>

4.16 临时字段（Temporary Field）

```
public class Account {
    private String username;
    private String cardNumber;
    private double balance;
    private double tempBalance; // 临时字段

    public void calculateInterest() {
        tempBalance = balance * 1.05; // 仅在计算利息时使用
        // 其他逻辑
    }
}
```

假设 Account 类包含临时字段 tempBalance 仅用于特定计算：

临时字段（Temporary Field）是指在类中仅用于特定功能或场景的实例变量，其存在使得类的职责变得模糊，影响代码可读性和可维护性。

4.16 临时字段（Temporary Field）

```
public class Account {
    private String username;
    private String cardNumber;
    private double balance;

    public InterestCalculator calculateInterest() {
        return new InterestCalculator(balance);
    }
}
```

```
class InterestCalculator {
    private double tempBalance;

    public InterestCalculator(double balance) {
        this.tempBalance = balance * 1.05;
    }
    // 其他计算逻辑
}
```

提炼类（Extract Class）：将临时字段及其相关操作封装到新类中，提升代码结构清晰度。

4.16 临时字段（Temporary Field）

临时字段 Temporary Field	**概念**	定义	某个实例变量仅为代码中一**小部分功能**临时所用而创建	
		影响	通常一个对象会需要它全部的变量。当一个变量看上去没什么用，却要试图了解它为什么在那里时，会使类的作用变得更**难理解，影响了代码的可读和可维护性**	
		目标	**消除临时字段**，提升代码可读性、可维护性	
		改进方法	**提炼类**，封装这些变量和需要他们的方法为方法对象、**引入特例**，为变量无效时创建额外组件，消除条件代码	
	症状	某个实例变量仅为某种情况而设置		
		某些实例变量仅为某个函数的复杂算法少传参数而设置		
	改进手法（仅列主要方法）	<<操作入口>>	快捷键（推荐）	Ctrl+Alt+Shift+T（或：鼠标右键“Refactor”、工具栏“Refactor”）
		提取类	N/A	Extract Delegate
		提取方法	Ctrl+Alt+M	Extract Method
		移除中间人	N/A	Remove Middleman
		安全删除	Alt+Delete	Safe Delete
		内联	Ctrl+Alt+N	Inline xxx
	排查方法	人工判断	分析变量或字段的生命周期	

4.17 过长的消息链（Message Chains）

- objectA.getObjectB().getObjectC().getValue()

- 定义：如果你看到用户向一个对象请求另一个对象，然后再向后者请求另一个对象，然后再请求另一个对象……这就是消息链。在实际代码中你看到的可能是一长串取值函数或一长串临时变量。
- 影响：客户端代码将与查找过程中的调用结构紧密耦合。一旦对象间的关系发生任何变化，客户端就不得不做出相应修改。
- 改进目标：针对过长消息链，可以用这时候应该使用隐藏委托关系，把调用链这种耦合关系放在中间人中。

4.17 过长的消息链（Message Chains）

<table>
<tr><td rowspan="10">过长的消息链
(Message Chains)</td><td rowspan="6">概念</td><td>定义</td><td colspan="2">如果你看到用户向一个对象请求另一个对象，然后再向后者请求另一个对象，然后再请求另一个对象……这就是消息链。
在实际代码中你看到的可能是一长串取值函数或一长串临时变量。
注：不是所有的消息链都是坏味道。需要自己甄别。</td></tr>
<tr><td>影响</td><td colspan="2">客户端代码将与查找过程中的调用结构紧密耦合。一旦对象间的关系发生任何变化，客户端就不得不做出相应修改。</td></tr>
<tr><td>目标</td><td colspan="2">针对过长消息链，可以用这时候应该使用隐藏委托关系，把调用链这种耦合关系放在中间人中。</td></tr>
<tr><td rowspan="4">改进方法</td><td colspan="2">观察清楚消息链的调用业务逻辑。</td></tr>
<tr><td colspan="2">通过提炼函数把所有相同调用放在一个函数中</td></tr>
<tr><td colspan="2">搬移函数到对应的中间人类中（新增或者使用以前的类）</td></tr>
<tr><td colspan="2">替换这个函数到之前调用链。隐藏委托关系</td></tr>
<tr><td rowspan="3">症状</td><td colspan="3">外部调用内部对象的使用层级委托的消息链。</td></tr>
<tr><td colspan="3">调用消息链被外部依赖，对外暴露形成了耦合。</td></tr>
<tr><td colspan="3">本质上没有形成深模块的设计。</td></tr>
<tr><td rowspan="4">改进手法
(仅列主要方法)</td><td><<操作入口>></td><td>快捷键（推荐）</td><td>Ctrl+Alt+Shift+T（或：鼠标右键“Refactor”、工具栏“Refactor”）</td></tr>
<tr><td>提炼函数</td><td>Ctrl + Alt + M</td><td>Extract Method</td></tr>
<tr><td>移动函数</td><td>F6</td><td>Move Method</td></tr>
<tr><td>重命名</td><td>Shift + F6</td><td>Rename</td></tr>
<tr><td>辅助工具/参考资料</td><td>代码怀味道+重构 学习资料</td><td colspan="2">https://refactoring.guru/</td></tr>
</table>

4.18 中间人（Middle Man）

- 定义：一种过度使用委托（某类中一半以上方法都委托给其他类）的代码[1]
- 影响：当需求发生某些变化的时候，作为中间人的代码总会被牵连一并修改，代码越发臃肿
- 改进目标：减少委托
- 方法：移除中间人、内联、以继承取代委托（旧版）、以委托取代超类/子类（新版）

4.18 中间人（Middle Man）

中间人 Middle Man	**概念**	定义	一种**过度使用委托**（某类中**一半以上方法**都委托给其他类）的代码	
		影响	当需求发生某些变化的时候，作为中间人的代码总会**被牵连一并修改**，代码越发**臃肿**	
		目标	**减少委托**	
		改进方法	**移除中间人**；**内联**；以继承取代委托（旧版）；以委托取代超类/子类（新版）	
	症状	代码中委托过多，代码臃肿，增加后期修改成本		
		代码被委托类中含有“不干实事”的方法		
	改进手法（仅列主要方法）	<<操作入口>>	快捷键（推荐）	Ctrl+Alt+Shift+T（或：鼠标右键“Refactor”、工具栏“Refactor”）
		移除中间人	N/A	Remove Middleman
		用内联移除	Ctrl+Shift+N	Inline Method
	排查方法	人工判断	查看委托方法数量及具体实现	
	辅助工具/参考资料	IDEA-Analyze-Inspect Code	可辅助识别中间人问题	

4.19 内幕交易（Insider Trading）

- 定义：模块之间互相引用，私下直接进行大量的数据访问和交换
- 影响：增大模块间的耦合，容易导致循环依赖，加快架构腐化，甚至会朝着大泥球式的架构发展，严重影响可维护性
- 改进目标：消除模块间不合理的依赖关系（特别是循环依赖），将私下的数据访问和交换放到明面上，使模块间解耦，提高可维护性
- 方法：搬移函数、搬移字段、隐藏委托关系、以委托取代子类/超类

4.19 内幕交易（Insider Trading）

内幕交易 Insider Trading	**概念**	定义	模块之间**互相引用**，**私下**直接进行大量的**数据访问和交换**	
		影响	增大模块间的**耦合**，容易导致**循环依赖**，加快**架构腐化**，甚至会朝着**大泥球**式的架构发展，严重影响可维护性	
		目标	消除模块间不合理的依赖关系（特别是循环依赖），**将私下的数据访问和交换放到明面上**，使模块间**解耦**，提高可维护性	
		改进方法	**搬移函数，搬移字段，隐藏委托关系，以委托取代子类/超类**	
	症状	类间存在不合理的继承关系		
		模块间存在循环依赖		
		模块间业务边界不清晰，随时有引入循环依赖的风险		
	改进手法（仅列主要方法）	<<操作入口>>	快捷键（推荐）	Ctrl+Alt+Shift+T（或：鼠标右键“Refactor”、工具栏“Refactor”）
		封装字段	N/A	Excapsulate Fields
		委托取代继承	N/A	Replace Inheritance with Delegation
		移除中间人	N/A	Remove Middleman
		实例方法变为静态方法	N/A	Make Static
		静态方法变为实例方法	N/A	Convert To Instance Method
		搬移实例/静态成员	F6	Move Instance Method/Move Static Members
		内联	Ctrl+Alt+N	Inline XXX
		提炼方法	Ctrl+Alt+M	Extract Method
	排查方法	通过依赖矩阵分析依赖关系	IDEA——Analyze——Analyze dependency Matrix Scope：识别代码中不合理的反向依赖（**对角线右上角出现数字即表示模块间依赖关系存在问题**），可辅助识别内幕交易等代码坏味道	
		通过UML类图辅助分析	选中文件或package——右键——Diagrams——show Diagram，生成类图	
		人工排查	对于工具无法识别、或代码无依赖但业务逻辑有耦合等场景，需要进行人工排查	
	辅助工具/参考资料	ArchUnit	对于新增或已重构“干净”的架构和分层，可以使用ArchUnit进行架构看护	
		《重改-改善既有代码设计》第2版	3.19章节	

4.20 过大的类（Large Class）

- 定义：由于属性未分组和职责不单一而包含过多属性、方法和代码行的类
- 影响：随着属性、方法和代码行数的不断增加，重复代码接踵而至，最终走向混乱
- 改进目标：拆分过大的类，确保类职责单一
- 方法：
  - 提取类
  - 提取子类
  - 提取接口/超类

4.20 过大的类（Large Class）

过大的类 Large Class	**概念**	定义	类想做的事情太多，包含过多的字段，方法，代码行	
		影响	代码重复、混乱，实例过多，进而导致其他代码坏味道	
		目标	分解类，保证类的职责单一	
		改进方法	提取类、提取子类、提取接口/超类	
	症状	类包含太多的字段、方法		
		类职责不单一		
		很多属性和方法只在特殊场景中使用		
	改进手法	<<操作入口>>	快捷键（推荐）	Ctrl+Alt+Shift+T（或：鼠标右键“Refactor”、工具栏“Refactor”）
		将参数提取为类	N/A	Introduce Parameter Object
		提取属性	Ctrl+Alt+F	Introduce Field
		提取函数	Ctrl+Alt+M	Extract Method
		方法下移	N/A	Push Members Down
		方法上移	N/A	Push Members Up
		提取接口	N/A	Extract Interface
		提取超类	N/A	Extract Superclass
		工厂替换构造函数	N/A	Replace Constructor With Factory Method

4.21 异曲同工的类 （alternative classes with different interfaces）

- 定义: 异曲同工的类——两个类功能一致，却有不同的定义(方法/接口)
- 影响: 相关的业务逻辑可能会重复实现，或分布到不同类中，代码难维护
- 改进目标: 统一接口、相同的功能只实现一份
- 方法: 函数改名、搬移函数、添加参数、函数参数化、提炼超类、移除子类

- 注：
  - 狭义的异曲同工：两个类功能完全相同；
  - 广义的异曲同工：两个类有很多相同的功能，从抽象角度看它们也有高一致性，却没有共同的基类/接口。

4.21 异曲同工的类 （alternative classes with different interfaces）

异曲同工的类 **alternative classes with different interfaces**	**概念**	定义	两个类功能一致，却有不同的定义(方法/接口)	
		影响	相关的业务逻辑可能会重复实现，或分布到不同类中，代码难维护	
			某个逻辑的修改需要修改多处，容易滋生重复代码、霰弹式修改	
		目标	统一接口、相同的功能只实现一份	
		改进方法	函数改名、上移函数、添加参数、函数参数化、提炼超类、移除子类	
	症状	类名、函数名类似		
		大部分逻辑一样		
		存在少量差异		
	改进手法	<<操作入口>>	快捷键（推荐）	Ctrl+Alt+Shift+T（或：鼠标右键“Refactor”、工具栏“Refactor”）
		改函数名	Shift+F6	Rename
		改函数签名 （含：改名、改参数类型与顺序）	Ctrl+F6	Change Signature
		搬移函数	F6	Move Instance Method
		提取超类	N/A	Extract Superclass
		上移函数、下移函数	N/A	Pull Members Up / Push Members Down
		内联函数	Ctrl+Shift+N	Inline Method
		删除函数	Alt+Del	Safe Delete
	排查方法	CodeCheck，IDEA->Analyze+人工判断	部分代码片段可能出现重复的，可以用重复代码工具识别出来，大部分时间需要人工判断	
	辅助工具/参考资料	IDEA-Analyze-Inspect Code	Simian/PMC-CMD/CloneDR/CCCD/CCFinder/Bauhaus/CodePro	

4.22 纯稚的数据类（Data Class）

- 定义: DataClass是指：它们拥有一些字段，以及用于访问（读写）这些字段的函数，除此之外一无长物。这样的类只是一种不会说话的数据容器。
- 影响: DataClass将数据与行为间关系割裂，破坏了面向对象的精髓。特定数据的操作分散在代码各处，可能造成代码霰弹式修改
- 改进目标: 将特定数据的操作集中在一个地方，提高代码的可读性、可维护性、可扩展性等
- 方法: 封装变量、封装集合、搬移函数、抽取方法、移除设值函数

4.22 纯稚的数据类（Data Class）

<table>
<tr><td rowspan="11">纯稚的数据类
(Data Class)</td><td rowspan="4">概念</td><td colspan="2">定义</td><td colspan="2">仅拥有一些字段，以及用于访问（读写）这些字段的函数，除此之外一无长物</td></tr>
<tr><td rowspan="2">影响</td><td colspan="2">将数据与行为间关系割裂，破坏了面向对象的精髓</td></tr>
<tr><td colspan="2">特定数据的操作分散在代码各处，可能造成代码霰弹式修改</td></tr>
<tr><td>目标</td><td colspan="2">将特定数据的操作集中在一个地方，提高代码的可读性、可维护性、可扩展性等</td></tr>
<tr><td>症状</td><td colspan="4">改进方法</td><td>封装变量、移除设值函数、封装集合、抽取方法、搬移函数等</td></tr>
<tr><td>症状</td><td colspan="5">一个类中仅有变量及Set/Get方法</td></tr>
<tr><td rowspan="7">改进手法
(仅列主要方法)</td><td colspan="2"><<操作入口>></td><td>快捷键（推荐）</td><td colspan="2">Ctrl+Alt+Shift+T（或：鼠标右键“Refactor”、工具栏“Refactor”）</td></tr>
<tr><td colspan="2">封装变量</td><td>Ctrl+Alt+Shift+T->Encapsulate Fields</td><td colspan="2">Encapsulate Fields</td></tr>
<tr><td colspan="2" rowspan="2">提取函数</td><td>Ctrl+Atl+M</td><td colspan="2" rowspan="2">Extract Method</td></tr>
<tr><td>Ctrl+Alt+Shift+T->Extract Method</td></tr>
<tr><td rowspan="2">移除设值函数</td><td>用内联移除</td><td>Ctrl+Alt+N</td><td colspan="2">Inline Method</td></tr>
<tr><td>直接删除</td><td>Alt+Del</td><td colspan="2">Safe Delete</td></tr>
<tr><td colspan="2" rowspan="2">搬移函数</td><td>F6</td><td colspan="2" rowspan="2">Move Instance Method</td></tr>
<tr><td>Ctrl+Alt+Shift+T->Move Instance Method</td></tr>
<tr><td>排查方法</td><td colspan="2">搜索+人工判断</td><td colspan="3">ArchGuarding或Infusion等工具，可对data class进行检测，但检测结果还需要人工进行判断</td></tr>
<tr><td>辅助工具/参考资料</td><td colspan="2">《重改-改善既有代码设计》第2版</td><td colspan="3">3.22节</td></tr>
</table>

4.23 注释（Comments）

- 定义:
  - 1.某段代码有大段的注释，这些注释之所以存在是因为代码很糟糕
  - 2.代码注释错误，或者缺少必要的注释
- 影响: 以注释掩盖了代码坏味道，或者没有正确的注释，不便阅读。影响了代码的可读、可维护性
- 改进目标: 消除不必要的注释，补充必要的注释，提升代码可读可维护性
- 方法
  - 1.提取变量（代码段注释）
  - 2.提取方法（代码段注释）
  - 3.方法/变量改名（方法/变量注释）
  - 4.补充/修改必要的注释

4.23 注释（Comments）

注释 Comments	**概念**	定义	某段代码有**大段的注释**，这些注释之所以存在是因为代码很糟糕 代码**注释错误**，或**缺少必要的注释** [1]	
		影响	以注释**掩盖了代码坏味道**，或者没有正确的注释，**不便阅读**。影响了代码的可读、可维护性	
		目标	**消除**不必要的注释，**补充**必要的注释，提升代码可读可维护性	
		改进方法	**提取变量**（代码段的注释） **提取方法**（代码段的注释） **方法/变量改名**（方法/变量的注释） **补充/修改必要的注释** [2]	
	改进手法	<<操作入口>>	快捷键（推荐）	Ctrl+Alt+Shift+T（或：鼠标右键“Refactor”、工具栏“Refactor”）
		提取变量	Ctrl+Alt+V	Introduce Variable
		提取方法	Ctrl+Alt+M	Extract Method
		方法/变量改名	Shift+F6	Rename
	辅助工具/参考资料	CodeCheck	可辅助识别注释使用规范性问题	
		IDEA-Analyze	IDEA自带功能，扫描项目错误Javadoc、拼写错误等	

4.24 被拒绝的遗赠（Refused Bequest）

- 定义: 对于某个子类，它只想继承基类的部分函数和数据，不需要基类提供的全部内容，这些不需要的内容就成为了子类的负担
- 影响: 这种坏味道通常影响并不大，但如果子类拒绝实现部分接口或者基类的方法只适用于某个子类特定的方法，就会对可维护、可扩展性等造成较大影响。
- 改进目标: 改进不合理的继承体系，使代码结构清晰、可控。
- 方法:
  - 函数/字段下移，让超类只持有子类共享的东西[1]
  - 以委托取代超类/子类

4.24 被拒绝的遗赠（Refused Bequest）

被拒绝的遗赠 refused-bequest	**概念**	定义	有一个子类，它想继承基类的部分函数和数据，**不需要基类提供的全部内容**，这些不需要的函数和数据成为子类实现的负担，就是被拒绝的遗赠。	
		影响	这种坏味道通常影响并不大，但是对于**子类拒绝实现的接口**或者基类的方法**只属于某个子类特定的方法**等，影响就比较大。	
		目标	改进继承体系设计，去除坏味道的继承方式，改用其他形式	
		改进方法	**函数下移、字段下移**，让超类只持有所有子类共享的东西[1]	
			以委托取代超类/子类	
	症状	子类不需要父类的所有函数和属性，还要实现不需要的抽象方法		
		不满足继承关系，但用继承获取相关属性		
	改进手法（仅列主要方法）	<<操作入口>>	快捷键（推荐）	Ctrl+Alt+Shift+T（或：鼠标右键“Refactor”、工具栏“Refactor”）
		接口/函数/字段下移	N/A	Push members down
		内联	Ctrl+Alt+N	Inline XXX
		引入变量	Ctrl+Alt+V	Introduce Variable
		引入参数	Ctrl+Alt+P	Introduce Parameter
		以委托取代继承	N/A	Replace inheritance with delegation
	排查方法	人工判断	查看类之间逻辑结构，排查	
	辅助工具/参考资料	《重构-改善既有代码的设计》	第三章 3.23节	

The end

December 23, 2025